Bazele inteligenței artificiale

Laborator – Grigore Ionuț

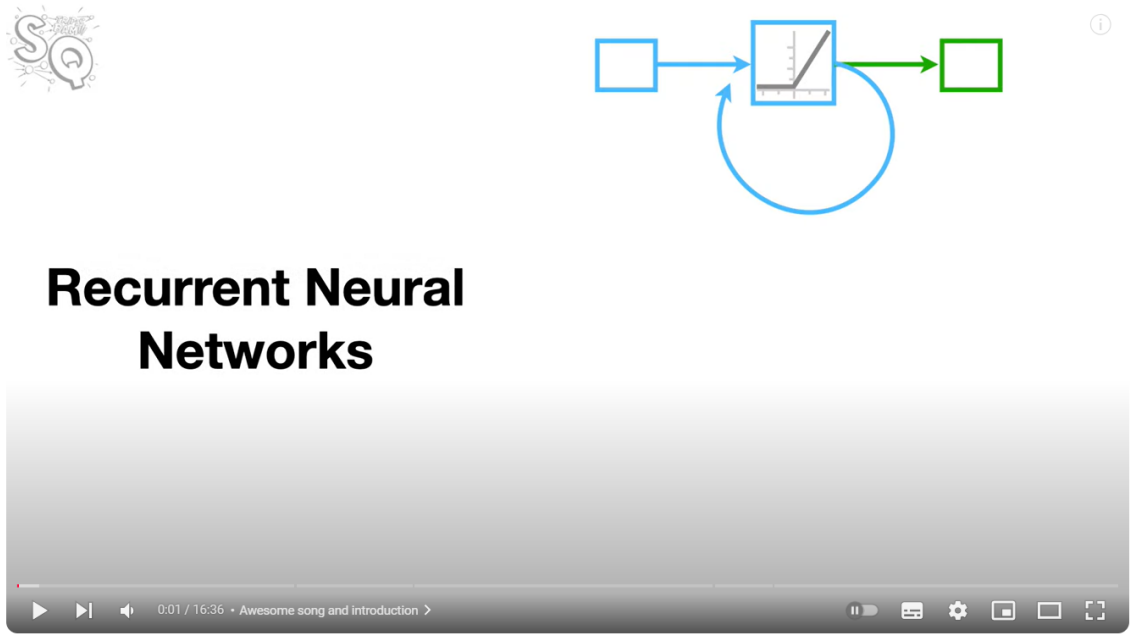
Laboratory contents

1. Introduction to Python
2. Linear models
3. Nonparametric models
4. Decision trees. Ensemble methods
5. Dimensionality reduction
6. Clustering
7. Test 1
8. Introduction to PyTorch
9. Multilayer perceptrons
10. Convolutional neural networks
11. Recurrent neural networks → **today's lab**
12. Attention mechanisms
13. Test 2
14. Recoveries

Content

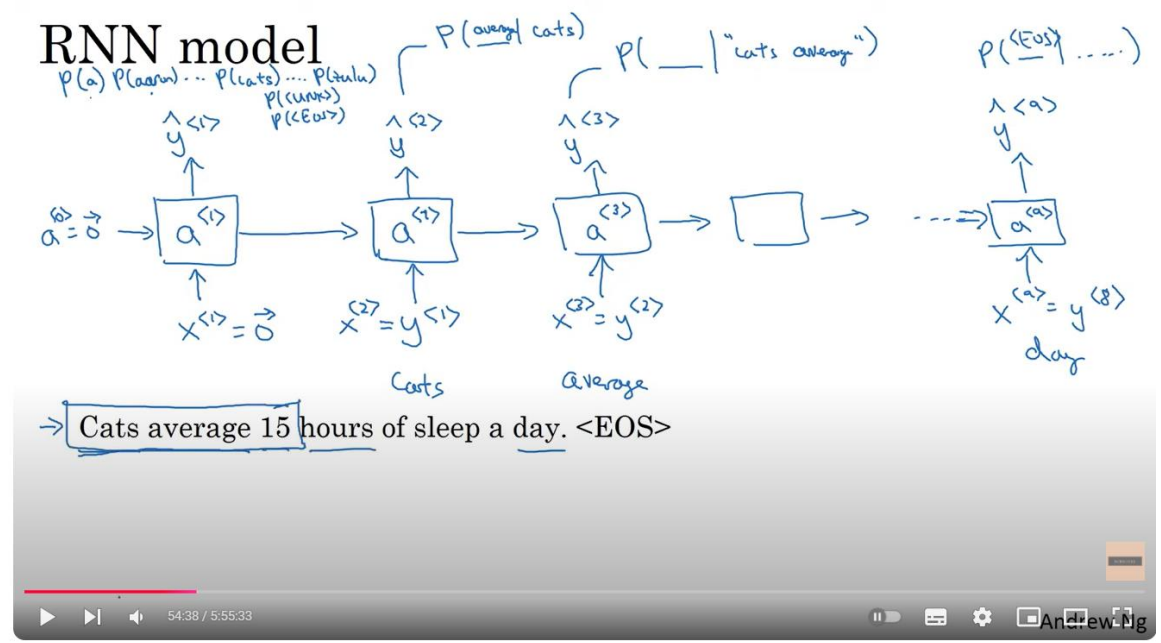
- Recurrent neural networks
- Prepocessing steps for text in PyTorch
- Gated recurrent units (GRU)
- Long short-term memory (LSTM)
- Sequence to sequence learning

1) Recurrent neural networks



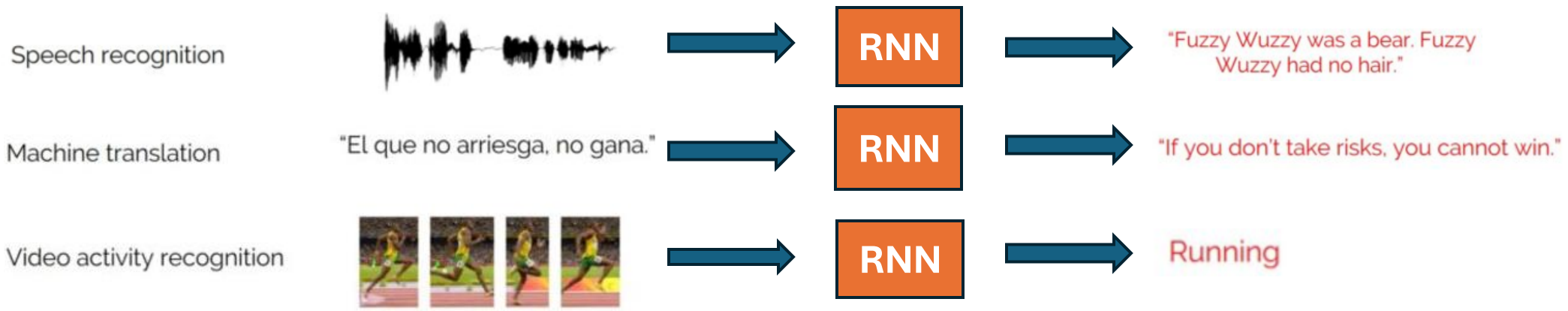
Recurrent Neural Networks (RNNs), Clearly Explained!!!

StatQuest with Josh Starmer
1,29 mil. de abonați
Alătură-te
Abonează-te
14 K
Trimite
Descarcă
Mulțumire



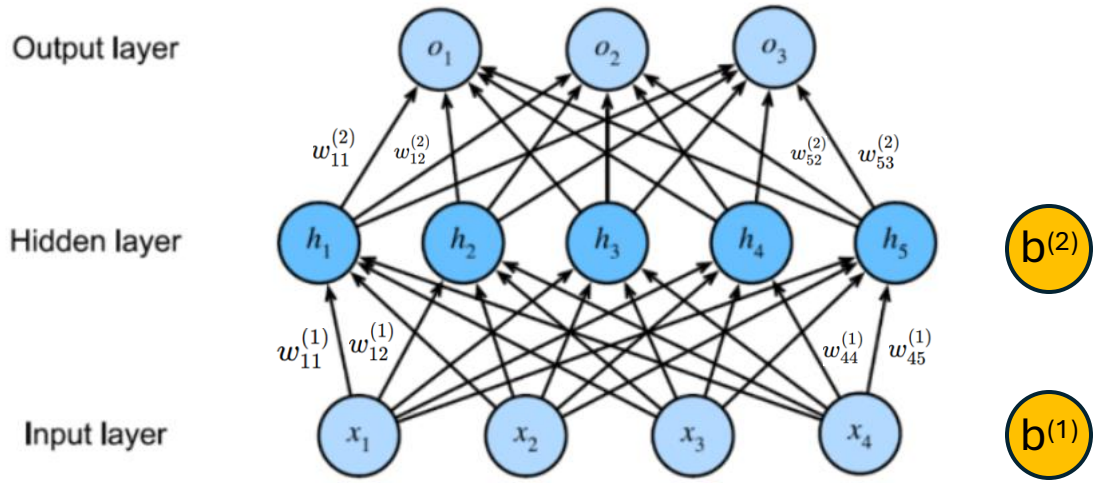
Sequence Models Complete Course

- **Recurrent neural networks (RNNs)** are designed to work with **sequential information** (time series, speech, text, audio, video, etc.):



- RNNs introduce **state variables** **to store past information**, together with the current inputs, to determine the current outputs:

Remember this example from lab 8 – a simple MLP



$$h_1 = \sigma\left(w_{11}^{(1)}x_1 + w_{21}^{(1)}x_2 + w_{31}^{(1)}x_3 + w_{41}^{(1)}x_4 + b_1^{(1)}\right)$$

$$h_2 = \sigma\left(w_{12}^{(1)}x_1 + w_{22}^{(1)}x_2 + w_{32}^{(1)}x_3 + w_{42}^{(1)}x_4 + b_2^{(1)}\right)$$

$$h_3 = \sigma\left(w_{13}^{(1)}x_1 + w_{23}^{(1)}x_2 + w_{33}^{(1)}x_3 + w_{43}^{(1)}x_4 + b_3^{(1)}\right)$$

$$h_4 = \sigma\left(w_{14}^{(1)}x_1 + w_{24}^{(1)}x_2 + w_{34}^{(1)}x_3 + w_{44}^{(1)}x_4 + b_4^{(1)}\right)$$

$$h_5 = \sigma\left(w_{15}^{(1)}x_1 + w_{25}^{(1)}x_2 + w_{35}^{(1)}x_3 + w_{45}^{(1)}x_4 + b_5^{(1)}\right)$$

$$o_1 = w_{11}^{(2)}h_1 + w_{21}^{(2)}h_2 + w_{31}^{(2)}h_3 + w_{41}^{(2)}h_4 + w_{51}^{(2)}h_5 + b_1^{(2)}$$

$$o_2 = w_{12}^{(2)}h_1 + w_{22}^{(2)}h_2 + w_{32}^{(2)}h_3 + w_{42}^{(2)}h_4 + w_{52}^{(2)}h_5 + b_2^{(2)}$$

$$o_3 = w_{13}^{(2)}h_1 + w_{23}^{(2)}h_2 + w_{33}^{(2)}h_3 + w_{43}^{(2)}h_4 + w_{53}^{(2)}h_5 + b_3^{(2)}$$

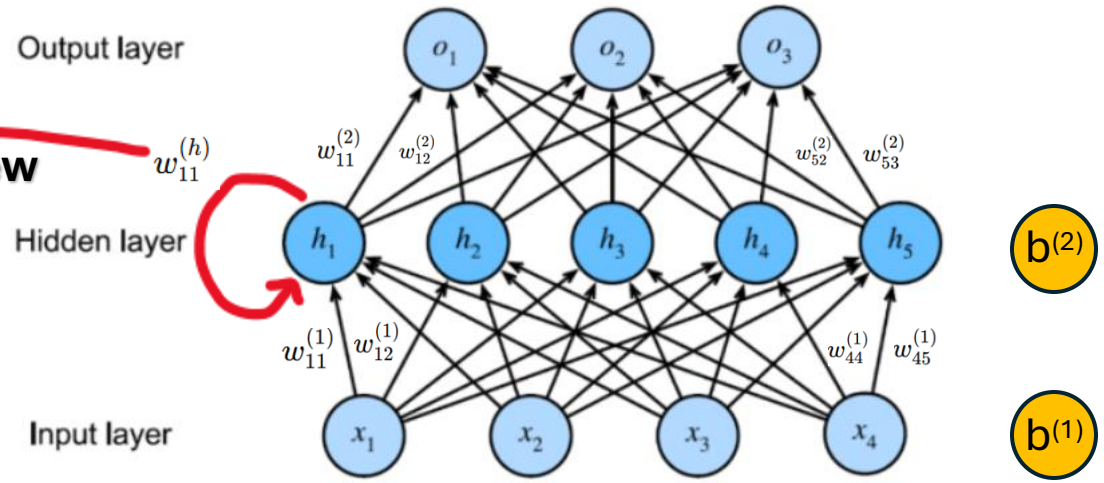
$$\mathbf{H} = \sigma(\mathbf{X}\mathbf{W}^{(1)} + \mathbf{b}^{(1)})$$

$$\mathbf{O} = \mathbf{H}\mathbf{W}^{(2)} + \mathbf{b}^{(2)}$$

We want to find these !

RNN – I know, a lot of math but is not so bad ☺

these are new



$$h_1^{(t)} = \sigma\left(w_{11}^{(1)}x_1^{(t)} + w_{21}^{(1)}x_2^{(t)} + w_{31}^{(1)}x_3^{(t)} + w_{41}^{(1)}x_4^{(t)} + \sum_{k=1}^{5} w_{k1}^{(h)}h_k^{(t-1)} + b_1^{(1)}\right)$$

$$h_2^{(t)} = \sigma\left(w_{12}^{(1)}x_1^{(t)} + w_{22}^{(1)}x_2^{(t)} + w_{32}^{(1)}x_3^{(t)} + w_{42}^{(1)}x_4^{(t)} + \sum_{k=1}^{5} w_{k2}^{(h)}h_k^{(t-1)} + b_2^{(1)}\right)$$

$$h_3^{(t)} = \sigma\left(w_{13}^{(1)}x_1^{(t)} + w_{23}^{(1)}x_2^{(t)} + w_{33}^{(1)}x_3^{(t)} + w_{43}^{(1)}x_4^{(t)} + \sum_{k=1}^{5} w_{k3}^{(h)}h_k^{(t-1)} + b_3^{(1)}\right)$$

$$h_4^{(t)} = \sigma\left(w_{14}^{(1)}x_1^{(t)} + w_{24}^{(1)}x_2^{(t)} + w_{34}^{(1)}x_3^{(t)} + w_{44}^{(1)}x_4^{(t)} + \sum_{k=1}^{5} w_{k4}^{(h)}h_k^{(t-1)} + b_4^{(1)}\right)$$

$$h_5^{(t)} = \sigma\left(w_{15}^{(1)}x_1^{(t)} + w_{25}^{(1)}x_2^{(t)} + w_{35}^{(1)}x_3^{(t)} + w_{45}^{(1)}x_4^{(t)} + \sum_{k=1}^{5} w_{k5}^{(h)}h_k^{(t-1)} + b_5^{(1)}\right)$$

$$o_1^{(t)} = w_{11}^{(2)}h_1^{(t)} + w_{21}^{(2)}h_2^{(t)} + w_{31}^{(2)}h_3^{(t)} + w_{41}^{(2)}h_4^{(t)} + w_{51}^{(2)}h_5^{(t)} + b_1^{(2)}$$

$$o_2^{(t)} = w_{12}^{(2)}h_1^{(t)} + w_{22}^{(2)}h_2^{(t)} + w_{32}^{(2)}h_3^{(t)} + w_{42}^{(2)}h_4^{(t)} + w_{52}^{(2)}h_5^{(t)} + b_2^{(2)}$$

$$o_3^{(t)} = w_{13}^{(2)}h_1^{(t)} + w_{23}^{(2)}h_2^{(t)} + w_{33}^{(2)}h_3^{(t)} + w_{43}^{(2)}h_4^{(t)} + w_{53}^{(2)}h_5^{(t)} + b_3^{(2)}$$

this is new

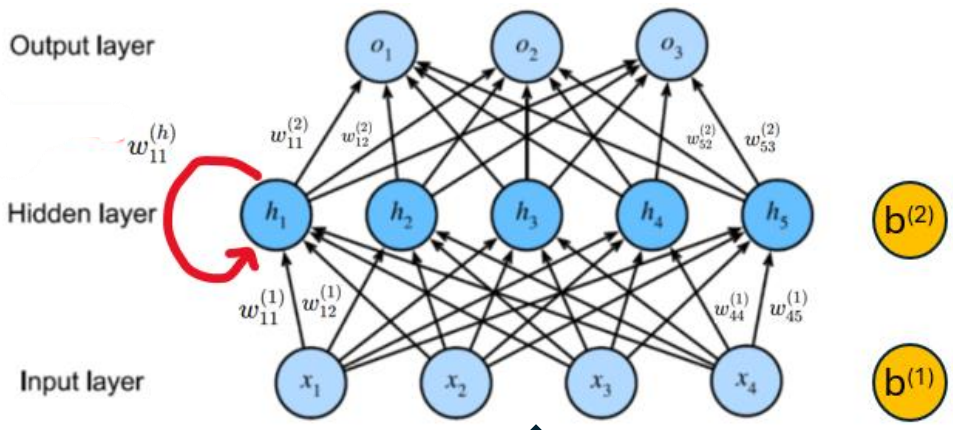
$$H^{(t)} = \sigma\left(X^{(t)}W^{(1)} + H^{(t-1)}W_h^{(1)} + b^{(1)}\right)$$

$$O^{(t)} = H^{(t)}W^{(2)} + b^{(2)}$$

We want to find these !

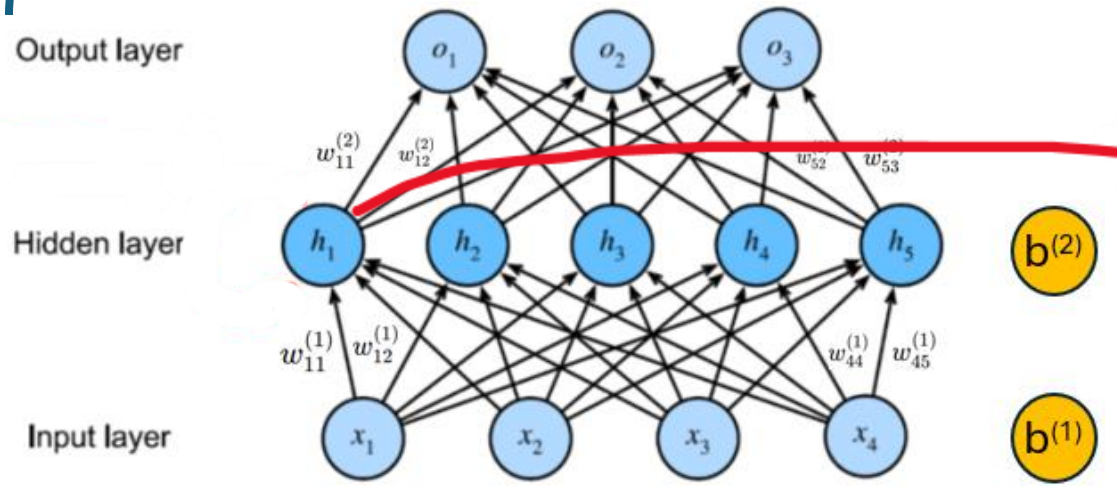
Unfolded form of an RNN

NOTE: here in this example I put just $w_{11}^{(h)}$ because otherwise it would make the diagram overly cluttered and harder to understand.

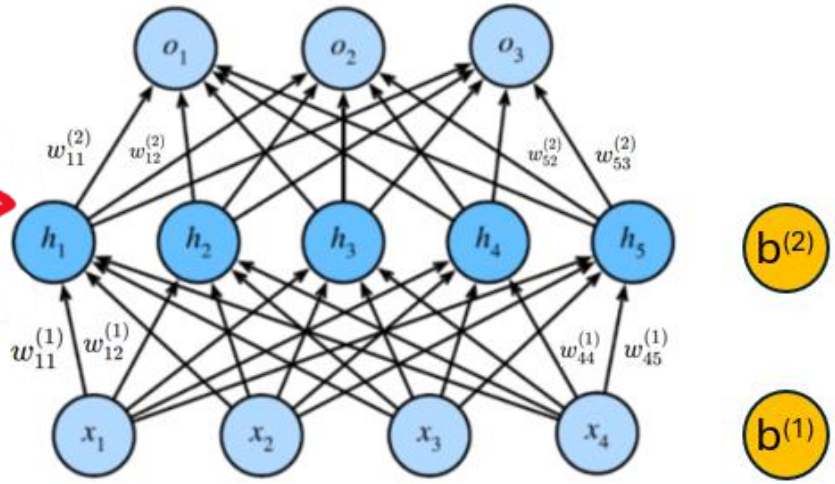


NOTE: we have 5 hidden units per hidden layer (h_1, h_2, h_3, h_4, h_5) ⇒ from one time step to other we will have 5x5 = 25 weights:

1. $w_{11}^{(h)}, w_{12}^{(h)}, w_{13}^{(h)}, w_{14}^{(h)}, w_{15}^{(h)}$
2. $w_{21}^{(h)}, w_{22}^{(h)}, w_{23}^{(h)}, w_{24}^{(h)}, w_{25}^{(h)}$
3. $w_{31}^{(h)}, w_{32}^{(h)}, w_{33}^{(h)}, w_{34}^{(h)}, w_{35}^{(h)}$
4. $w_{41}^{(h)}, w_{42}^{(h)}, w_{43}^{(h)}, w_{44}^{(h)}, w_{45}^{(h)}$
5. $w_{51}^{(h)}, w_{52}^{(h)}, w_{53}^{(h)}, w_{54}^{(h)}, w_{55}^{(h)}$



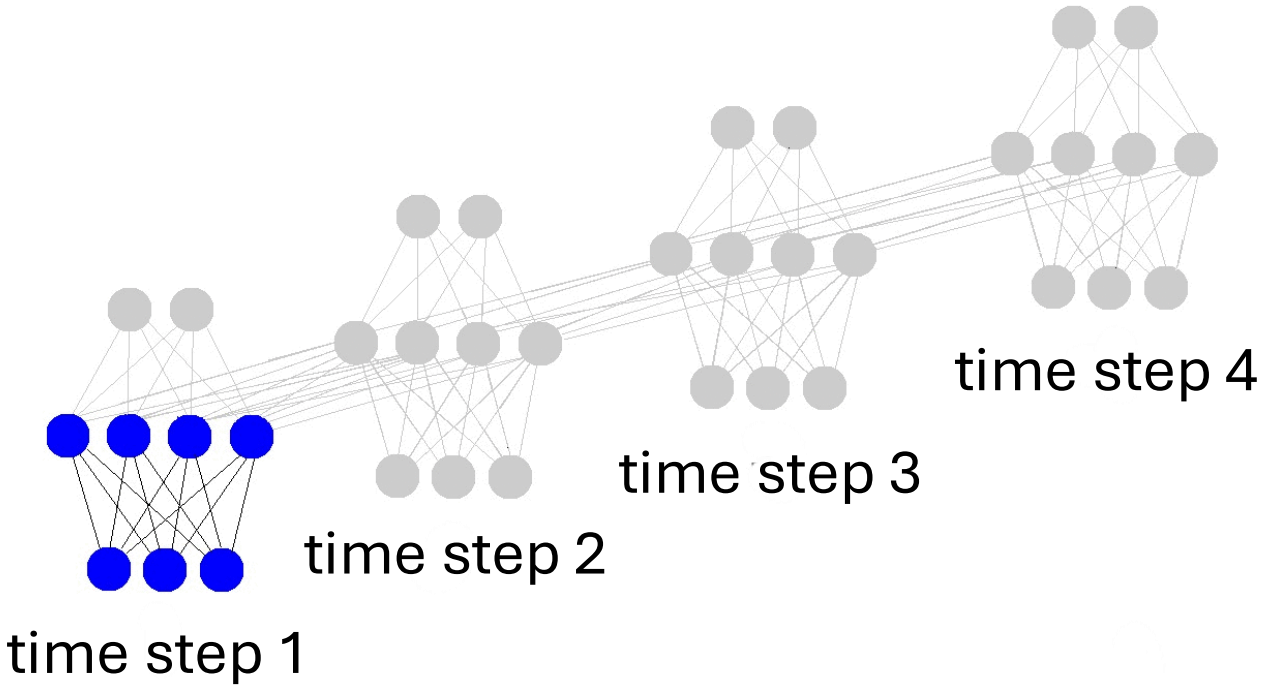
time step 1

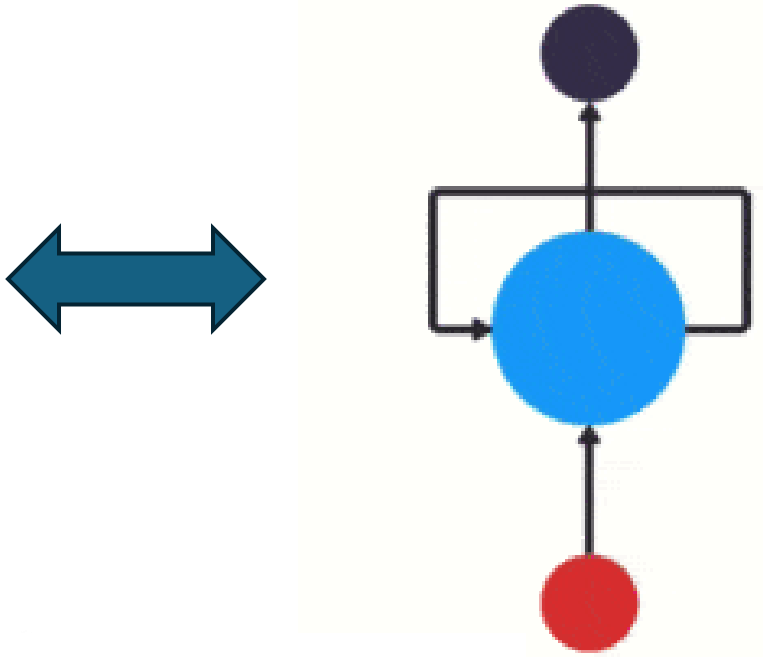


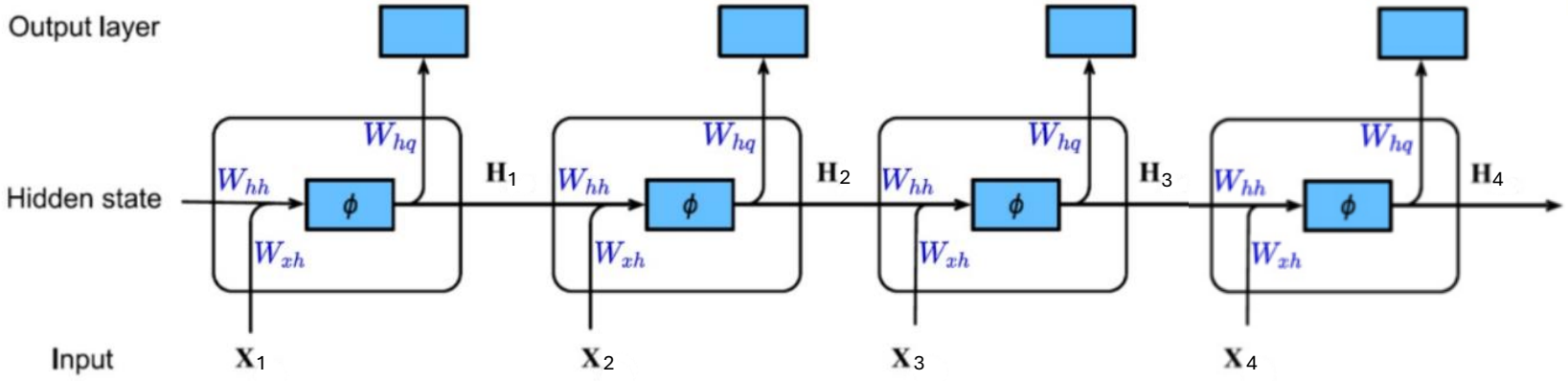
...and so on

time step 2

Unfolded form of an RNN



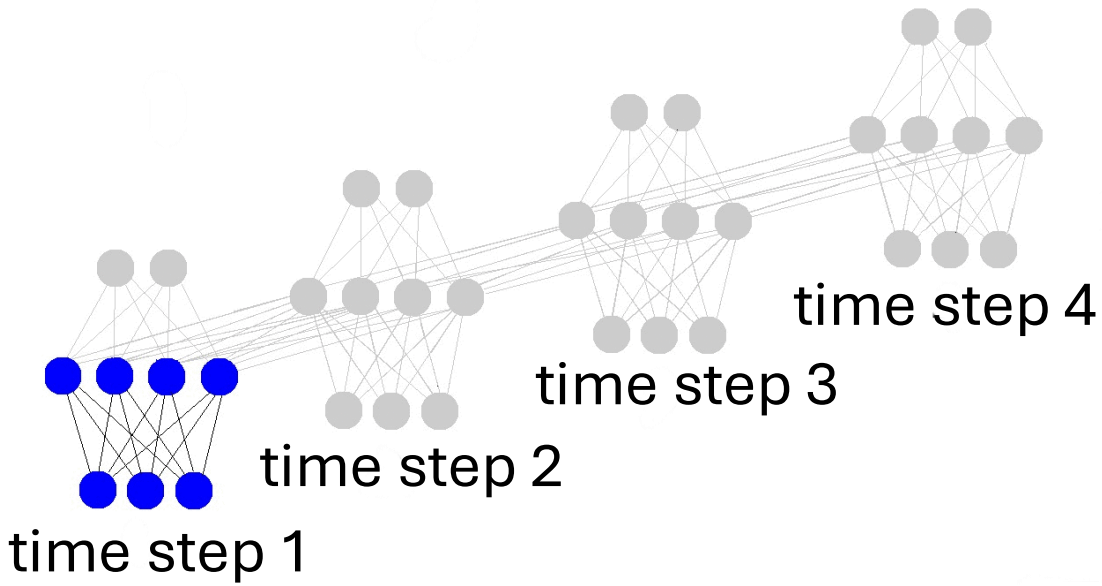


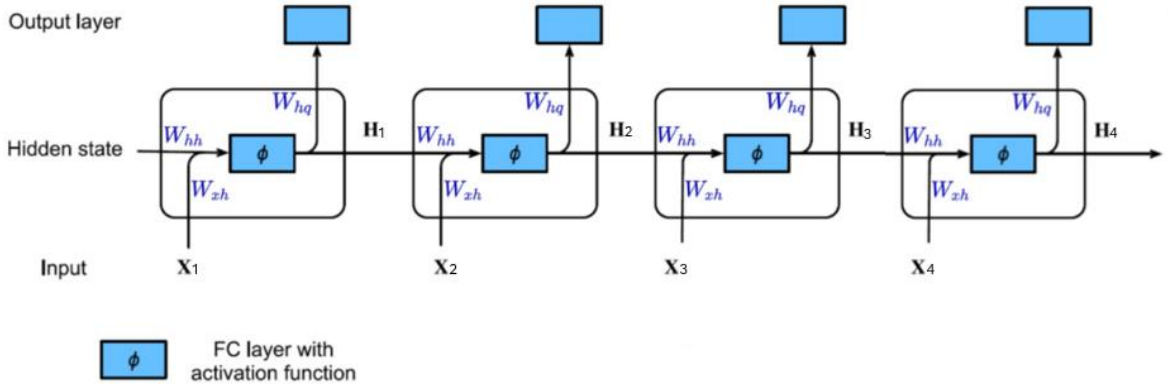


ϕ FC layer with activation function

VERY IMPORTANT

- **in an RNN, the weights and biases used for computing the hidden state and output are the same at every timestep**: input-to-hidden weights (W_{xh}), hidden-to-hidden (recurrent) weights (W_{hh}), hidden-to-output weights (W_{hq}) and biases (b)





```python
num_hiddens = 256
rnn_layer = nn.RNN(len(vocab), num_hiddens)
```

```python
class RNNModel(nn.Module):
    """The RNN model."""
    def __init__(self, rnn_layer, vocab_size, **kwargs):
        super(RNNModel, self).__init__(**kwargs)
        self.rnn = rnn_layer
        self.vocab_size = vocab_size
        self.num_hiddens = self.rnn.hidden_size
        self.linear = nn.Linear(self.num_hiddens, self.vocab_size)

    def forward(self, inputs, state):
        X = F.one_hot(inputs.T.long(), self.vocab_size)
        X = X.to(torch.float32)
        Y, state = self.rnn(X, state)
        # The fully connected layer will first change the shape of `Y` to
        # (`num_steps` * `batch_size`, `num_hiddens`). Its output shape is
        # (`num_steps` * `batch_size`, `vocab_size`).
        output = self.linear(Y.reshape((-1, Y.shape[-1])))
        return output, state

    def begin_state(self, device, batch_size=1):
        if not isinstance(self.rnn, nn.LSTM):
            # `nn.GRU` takes a tensor as hidden state
            return  torch.zeros((self.rnn.num_layers,
                                 batch_size, self.num_hiddens),
                                 device=device)
        else:
            # `nn.LSTM` takes a tuple of hidden states
            return (torch.zeros((
                self.rnn.num_layers,
                batch_size, self.num_hiddens), device=device),
                    torch.zeros((
                        self.rnn.num_layers,
                        batch_size, self.num_hiddens), device=device))
```

```python
net = RNNModel(rnn_layer, vocab_size=len(vocab))
```

2) Prepocessing steps for text in PyTorch

- **Preprocessing steps for text**:

1) Load text as strings into memory
2) Split strings into **tokens** (e.g. words and characters)
3) Build a table of vocabulary/dictionary to map the split tokens to numerical indices (starting from 0)
4) Convert text into sequences of numerical indices so they can be manipulated by RNNs easily

Load text as strings into memory

function from the lab, search it with CTRL + F ☺

```python
def read_time_machine():
    """Load the time machine dataset into a list of text lines."""
    with open(download('http://d2l-data.s3-accelerate.amazonaws.com/timemachine.txt'), 'r') as f:
        lines = f.readlines()
    return [re.sub('[^A-Za-z]+', ' ', line).strip().lower() for line in lines]
```

- *read_time_machine()* function reads the dataset into a list of text lines
- ***timemachine.text*** contains the full text of *The Time Machine*, a classic science fiction novel by H.G. Wells
- **NOTE 1**: each line is a string! (I know it's weird to say that a line is a string but look at the example below ☺)
- **NOTE 2**: - we removed non-alphabetical characters ([^A-Za-z]+) and replaces them with spaces
  - we striped extra whitespace and converts all text to lowercase (.lower())

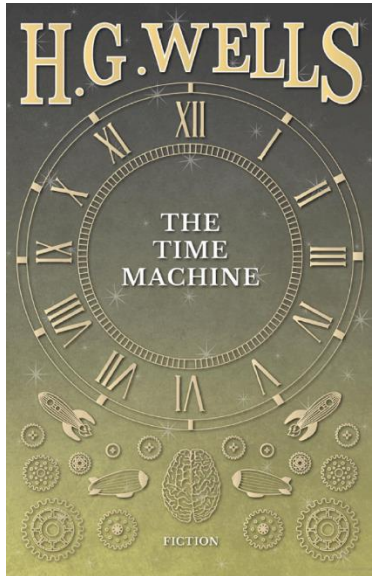
```python
lines = read_time_machine()
lines
```

```
['the time machine by h g wells',
 '',
 '',
 '',
 '',
 'i',
 '',
 '',
 'the time traveller for so it will be convenient to speak of him',
 'was expounding a recondite matter to us his grey eyes shone and',
 'twinkled and his usually pale face was flushed and animated the',
 'fire burned brightly and the soft radiance of the incandescent',
 'lights in the lilies of silver caught the bubbles that flashed and',
```

first line/string

second line/string

...and so on...



Split strings into **tokens** (e.g. words and characters)

```python
def tokenize(lines, token='word'):
    """Split text lines into word or character tokens."""
    if token == 'word':
        return [line.split() for line in lines]
    elif token == 'char':
        return [list(line) for line in lines]
    else:
        print('ERROR: unknown token type: ' + token)
```

- *tokenize()* function takes a list (lines) as the input and splits each text line into a list of **tokens**
- it supports tokenization at two levels: **word** and **character**. This is a common preprocessing step in natural language processing (NLP) tasks
- **word level:**

```python
tokens = tokenize(lines, token='word')
print(tokens[0])
```

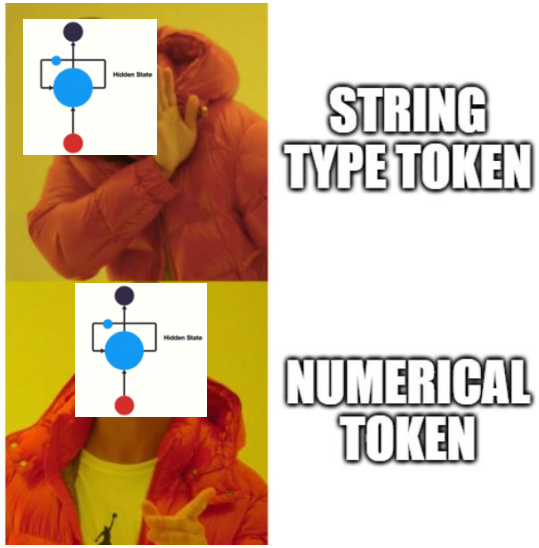
```
['the', 'time', 'machine', 'by', 'h', 'g', 'wells']
```

- **character level:**

```python
tokens = tokenize(lines, token='char')
print(tokens[0])
```

```
['t', 'h', 'e', ' ', 't', 'i', 'm', 'e', ' ', 'm', 'a', 'c', 'h', 'i', 'n', 'e', ' ', 'b', 'y', ' ', 'h', ' ', 'g', ' ', 'w', 'e', 'l', 'l', 's']
```

Build a table of vocabulary/dictionary to map the split tokens to numerical indices (starting from 0)



- the string type of the token is inconvenient to be used by models which take numerical inputs
- how we switch from string to numerical?

  a) count the unique tokens in the dataset (here dataset is often called **corpus**) and then assign a numerical index to each unique token according to its frequency

Example: suppose we have the following dataset:

```
"the quick brown fox jumps over the lazy dog"
"the quick dog jumps"
```

we first calculate how many times each token appears in the dataset:

```
'the': 3
'quick': 2
'jumps': 2
'dog': 2
'brown': 1
'fox': 1
'over': 1
'lazy': 1
```

Build a table of vocabulary/dictionary to map the split tokens to numerical indices (starting from 0)

we assign numerical indices to the sorted tokens, starting from 0 for unknown (**<unk>**) and including reserved tokens if needed:

Index	Token	
0	<unk>	# Reserved for unknown tokens
1	the	
2	quick	
3	jumps	
4	dog	
5	brown	
6	fox	
7	over	
8	lazy	

for example this one ☺

- rarely appeared tokens are often removed to reduce the complexity
- any token that does not exist in the dataset/corpus or has been removed is mapped into a special unknown token **<unk>**
- Other reserved tokens: - **<pad>** - padding - is used to pad sequences so that all input sequences have the same length. This is necessary because ML models often require inputs of fixed size for batch processing:

```
Sentence 1: "I love programming." (length = 3)
Sentence 2: "Deep learning is amazing!" (length = 4)
```

→

```
Padded Sentence 1: "I love programming <pad>"
Padded Sentence 2: "Deep learning is amazing!"
```

Build a table of vocabulary/dictionary to map the split tokens to numerical indices (starting from 0)

- **<bos>** - beginning of a sequence – it's used to mark the start of a sequence, indicating where the input begins:

```
Original Input: "Hello world"
Input with `<bos>`: "<bos> Hello world"
```

- **<eos>** - end of a sequence - it marks the end of a sequence, signaling the model to stop processing or generating further tokens:

```
Generated Output: "Good morning everyone <eos>"
```

Let's explain now step by step the Vocab class (see next slide)

Build a table of vocabulary/dictionary to map the split tokens to numerical indices (starting from 0)

```python
class Vocab:
    """Vocabulary for text."""
    def __init__(self, tokens=None, min_freq=0, reserved_tokens=None):
        if tokens is None:
            tokens = []
        if reserved_tokens is None:
            reserved_tokens = []
        # Sort according to frequencies
        counter = count_corpus(tokens)
        self._token_freqs = sorted(counter.items(), key=lambda x: x[1],
                                   reverse=True)
        # The index for the unknown token is 0
        self.idx_to_token = ['<unk>'] + reserved_tokens
        self.token_to_idx = {token: idx
                             for idx, token in enumerate(self.idx_to_token)}
        for token, freq in self._token_freqs:
            if freq < min_freq:
                break
            if token not in self.token_to_idx:
                self.idx_to_token.append(token)
                self.token_to_idx[token] = len(self.idx_to_token) - 1
```

- tokens:
  - a list (or list of lists) of tokens, such as words or characters. These tokens will be used to build the vocabulary.
  - default: None (empty vocabulary)
- min_freq:
  - minimum frequency threshold for tokens to be included in the vocabulary
  - default: 0 (include all tokens, regardless of frequency)
- reserved_tokens:
  - a list of tokens that are always included in the vocabulary, such as <pad>, <bos>, <eos>
  - default: None (no reserved tokens)

Build a table of vocabulary/dictionary to map the split tokens to numerical indices (starting from 0)

```python
    def __len__(self):
        return len(self.idx_to_token)

    def __getitem__(self, tokens):
        if not isinstance(tokens, (list, tuple)):
            return self.token_to_idx.get(tokens, self.unk)
        return [self.__getitem__(token) for token in tokens]

    def to_tokens(self, indices):
        if not isinstance(indices, (list, tuple)):
            return self.idx_to_token[indices]
        return [self.idx_to_token[index] for index in indices]

    @property
    def unk(self):  # Index for the unknown token
        return 0

    @property
    def token_freqs(self):  # Token frequencies
        return self._token_freqs

def count_corpus(tokens):
    """Count token frequencies."""
    # Here `tokens` is a 1D list or 2D list
    if len(tokens) == 0 or isinstance(tokens[0], list):
        # Flatten a list of token lists into a list of tokens
        tokens = [token for line in tokens for token in line]
    return collections.Counter(tokens)
```

- returns the size of the vocabulary (i.e., the total number of tokens)

- converts tokens (or a list/tuple of tokens) into their corresponding indices
- if a token is not found in the vocabulary, it returns the index of <unk>

- converts indices (or a list/tuple of indices) into their corresponding tokens

- returns the index of the unknown token (<unk>), which is always 0

- returns the list of token-frequency pairs sorted by frequency in descending order

- counts the frequency of tokens in the input, which can be either:
  - a 1D list of tokens (e.g., ['time', 'machine', 'time’])
  - a 2D list of token lists (e.g., [['time', 'machine'], ['time']])

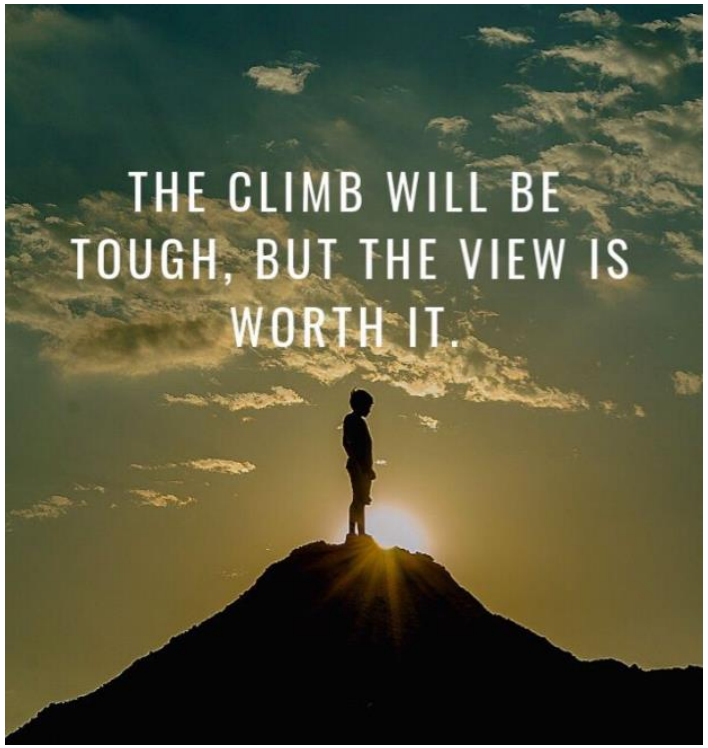
Build a table of vocabulary/dictionary to map the split tokens to numerical indices (starting from 0)

```
vocab = Vocab(tokens)

print(vocab.token_to_idx.items())
print(len(vocab))
```

```
dict_items([('<unk>', 0), ('the', 1), ('i', 2), ('and', 3), ('of', 4), ('a', 5), ('to', 6), ('was', 7),
4580
```

...and so on...



Convert text into sequences of numerical indices so they can be manipulated by RNNs easily

combine previous 3 steps

- flattens all the tokenized lines into a single list of tokens
- converts each token into its corresponding index using the vocabulary (vocab)
- optionally truncates the resulting list to a maximum number of tokens, specified by *max_tokens*

```python
def load_corpus_time_machine(max_tokens=-1):
    """Return token indices and the vocabulary of the time machine dataset."""
    lines = read_time_machine()
    tokens = tokenize(lines, 'char')
    vocab = Vocab(tokens)
    # Since each text line in the time machine dataset is not necessarily a
    # sentence or a paragraph, flatten all the text lines into a single list
    corpus = [vocab[token] for line in tokens for token in line]
    if max_tokens > 0:
        corpus = corpus[:max_tokens]
    return corpus, vocab
```

```python
corpus, vocab = load_corpus_time_machine()
vocab.token_to_idx.items()
```

```
dict_items([('<unk>', 0), (' ', 1), ('e', 2), ('t', 3), ('a', 4), ('i', 5), ('n', 6), ('o', 7), ('s', 8), ('h', 9), ('r', 10), ('d', 11), ('l', 12), ('m', 13), ('u', 14), ('c', 15), ('f', 16), ('w', 17), ('g', 18), ('y', 19), ('p', 20), ('b', 21), ('v', 22), ('k', 23), ('x', 24), ('z', 25), ('j', 26), ('q', 27)])
```

```python
corpus
```

```
[3,
 9,
 2,
 1,
 3,
 5,
 13,
 2,
 1,
```

```python
len(corpus)
```

```
170580
```

this is the dataset we will train our RNN with !

Just a few additional functions and we are ready

```python
def seq_data_iter_sequential(corpus, batch_size, num_steps):
    """Generate a mini-batch of subsequences using sequential partitioning."""
    # Start with a random offset to partition a sequence
    offset = random.randint(0, num_steps)
    num_tokens = ((len(corpus) - offset - 1) // batch_size) * batch_size
    Xs = torch.tensor(corpus[offset: offset + num_tokens])
    Ys = torch.tensor(corpus[offset + 1: offset + 1 + num_tokens])
    Xs, Ys = Xs.reshape(batch_size, -1), Ys.reshape(batch_size, -1)
    num_batches = Xs.shape[1] // num_steps
    for i in range(0, num_steps * num_batches, num_steps):
        X = Xs[:, i: i + num_steps]
        Y = Ys[:, i: i + num_steps]
        yield X, Y
```

```python
class SeqDataLoader:
    """An iterator to load sequence data."""
    def __init__(self, batch_size, num_steps, max_tokens):
        self.corpus, self.vocab = load_corpus_time_machine(max_tokens)
        self.batch_size, self.num_steps = batch_size, num_steps

    def __iter__(self):
        return seq_data_iter_sequential(self.corpus, self.batch_size, self.num_steps)
```

```python
def load_data_time_machine(batch_size, num_steps, max_tokens=10000):
    """Return the iterator and the vocabulary of the time machine dataset."""
    data_iter = SeqDataLoader(
        batch_size, num_steps, max_tokens)
    return data_iter, data_iter.vocab
```

these are specified in the problem

```python
batch_size, num_steps = 32, 35
train_iter, vocab = load_data_time_machine(batch_size, num_steps)
```

this will be the input for the RNN !

Recap

```python
def download(url, cache_dir=os.path.join('..', 'data')):
    """Download a file, return the local filename."""
    os.makedirs(cache_dir, exist_ok=True)
    fname = os.path.join(cache_dir, url.split('/')[-1])
    if os.path.exists(fname):
        with open(fname, 'rb') as f:
            while True:
                data = f.read(1048576)
                if not data:
                    break
        return fname
    print(f'Downloading {fname} from {url}...')
    r = requests.get(url, stream=True, verify=True)
    with open(fname, 'wb') as f:
        f.write(r.content)
    return fname
```

```python
def read_time_machine():
    """Load the time machine dataset into a list of text lines."""
    with open(download('http://d2l-data.s3-accelerate.amazonaws.com/timemachine.txt'), 'r') as f:
        lines = f.readlines()
    return [re.sub('[^A-Za-z]+', ' ', line).strip().lower() for line in lines]
```

```python
def tokenize(lines, token='word'):
    """Split text lines into word or character tokens."""
    if token == 'word':
        return [line.split() for line in lines]
    elif token == 'char':
        return [list(line) for line in lines]
    else:
        print('ERROR: unknown token type: ' + token)
```

```python
class Vocab:
    """Vocabulary for text."""
    def __init__(self, tokens=None, min_freq=0, reserved_tokens=None):
        if tokens is None:
            tokens = []
        if reserved_tokens is None:
            reserved_tokens = []
        # Sort according to frequencies
        counter = count_corpus(tokens)
        self._token_freqs = sorted(counter.items(), key=lambda x: x[1],
                                   reverse=True)
        # The index for the unknown token is 0
        self.idx_to_token = ['<unk>'] + reserved_tokens
        self.token_to_idx = {token: idx
                             for idx, token in enumerate(self.idx_to_token)}
        for token, freq in self._token_freqs:
            if freq < min_freq:
                break
            if token not in self.token_to_idx:
                self.idx_to_token.append(token)
                self.token_to_idx[token] = len(self.idx_to_token) - 1

    def __len__(self):
        return len(self.idx_to_token)

    def __getitem__(self, tokens):
        if not isinstance(tokens, (list, tuple)):
            return self.token_to_idx.get(tokens, self.unk)
        return [self.__getitem__(token) for token in tokens]

    def to_tokens(self, indices):
        if not isinstance(indices, (list, tuple)):
            return self.idx_to_token[indices]
        return [self.idx_to_token[index] for index in indices]

    @property
    def unk(self):  # Index for the unknown token
        return 0

    @property
    def token_freqs(self):  # Token frequencies
        return self._token_freqs

def count_corpus(tokens):
    """Count token frequencies."""
    # Here `tokens` is a 1D list or 2D list
    if len(tokens) == 0 or isinstance(tokens[0], list):
        # Flatten a list of token lists into a list of tokens
        tokens = [token for line in tokens for token in line]
    return collections.Counter(tokens)
```

```python
def load_corpus_time_machine(max_tokens=-1):
    """Return token indices and the vocabulary of the time machine dataset."""
    lines = read_time_machine()
    tokens = tokenize(lines, 'char')
    vocab = Vocab(tokens)
    # Since each text line in the time machine dataset is not necessarily a
    # sentence or a paragraph, flatten all the text lines into a single list
    corpus = [vocab[token] for line in tokens for token in line]
    if max_tokens > 0:
        corpus = corpus[:max_tokens]
    return corpus, vocab
```

```python
corpus, vocab = load_corpus_time_machine()
```

Recap

```python
def seq_data_iter_sequential(corpus, batch_size, num_steps):
    """Generate a mini-batch of subsequences using sequential partitioning."""
    # Start with a random offset to partition a sequence
    offset = random.randint(0, num_steps)
    num_tokens = ((len(corpus) - offset - 1) // batch_size) * batch_size
    Xs = torch.tensor(corpus[offset: offset + num_tokens])
    Ys = torch.tensor(corpus[offset + 1: offset + 1 + num_tokens])
    Xs, Ys = Xs.reshape(batch_size, -1), Ys.reshape(batch_size, -1)
    num_batches = Xs.shape[1] // num_steps
    for i in range(0, num_steps * num_batches, num_steps):
        X = Xs[:, i: i + num_steps]
        Y = Ys[:, i: i + num_steps]
        yield X, Y
```

```python
class SeqDataLoader:
    """An iterator to load sequence data."""
    def __init__(self, batch_size, num_steps, max_tokens):
        self.corpus, self.vocab = load_corpus_time_machine(max_tokens)
        self.batch_size, self.num_steps = batch_size, num_steps

    def __iter__(self):
        return seq_data_iter_sequential(self.corpus, self.batch_size, self.num_steps)
```

```python
def load_data_time_machine(batch_size, num_steps, max_tokens=10000):
    """Return the iterator and the vocabulary of the time machine dataset."""
    data_iter = SeqDataLoader(
        batch_size, num_steps, max_tokens)
    return data_iter, data_iter.vocab
```

these are specified in the problem

```python
batch_size, num_steps = 32, 35
train_iter, vocab = load_data_time_machine(batch_size, num_steps)
```

all the work for this ☺

download the dataset:

```
['the time machine by h g wells',
 '',
 '',
 '',
 '',
 'i',
 '',
 '',
 'the time traveller for so it will be convenient to speak of him',
 'was expounding a recondite matter to us his grey eyes shone and',
 'twinkled and his usually pale face was flushed and animated the',
 'fire burned brightly and the soft radiance of the incandescent',
 'lights in the lilies of silver caught the bubbles that flashed and',
```

tokenize each line at char or word-level:

```
['the', 'time', 'machine', 'by', 'h', 'g', 'wells']
```

```
['t', 'h', 'e', ' ', 't', 'i', 'm', 'e', ' ', 'm', 'a', 'c', 'h', 'i', 'n', 'e', ' ', 'b', 'y', ' ', 'h', ' ', 'g', ' ', 'w', 'e', 'l', 'l', 's']
```

build a table of vocabulary/dictionary (see slide 17):

[('<unk>', 0), (' ', 1), ('e', 2), ('t', 3), ('a', 4), ('i', 5), ('n', 6), ('o', 7), ('s', 8), ('h', 9), ('r', 10), ('d', 11), ('l', 12), ('m', 13), ('u', 14), ('c', 15), ('f', 16), ('w', 17), ('g', 18), ('y', 19), ('p', 20), ('b', 21), ('v', 22), ('k', 23), ('x', 24), ('z', 25), ('j', 26), ('q', 27)] → char-level

[('<unk>', 0), ('the', 1), ('i', 2), ('and', 3), ('of', 4), ('a', 5), ('to', 6), ('was', 7), ('in', 8), ('that', 9), ('my', 10), etc.] → word-level

map each token to its numerical index and create a single list with indices for the dataset:

char-level

```
[3,
 9,
 2,
 1,
 3,
 5,
 13,
 2,
 1,
 13,
 4,
 15,
 9,
```

word-level

```
[1,
 19,
 50,
 40,
 2183,
 2184,
 400,
 2,
 1,
 19,
 71,
 16,
 37,
```

we need one more step, just one ☺

we take the list with the dataset:

```
[3,
 9,
 2,
 1,
 3,
 5,
 13,
 2,
 1,
 13,
 4,
 15,
 9,
```

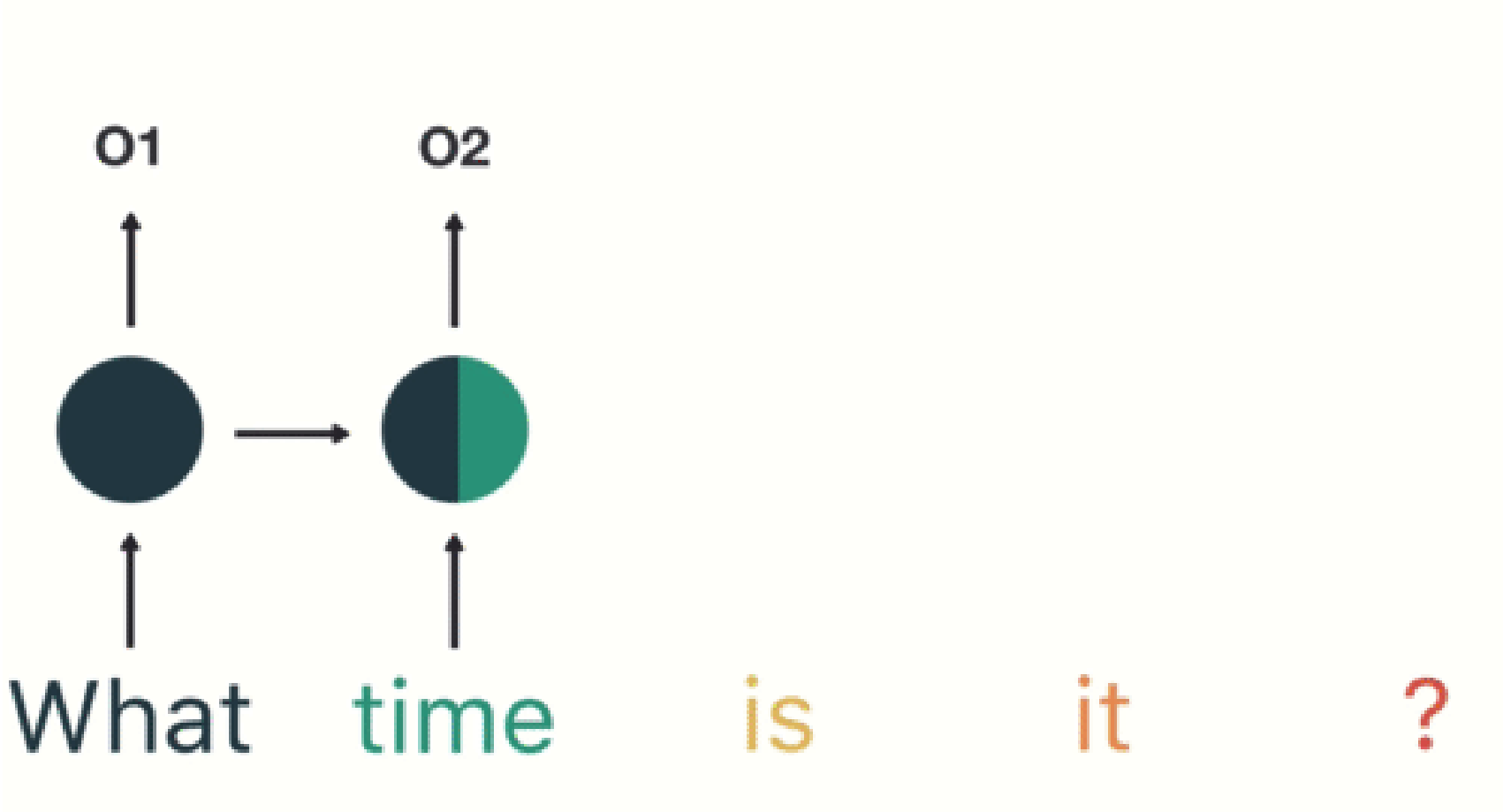
we need to do **one-hot encoding**!

Simple one-hot encoding example:

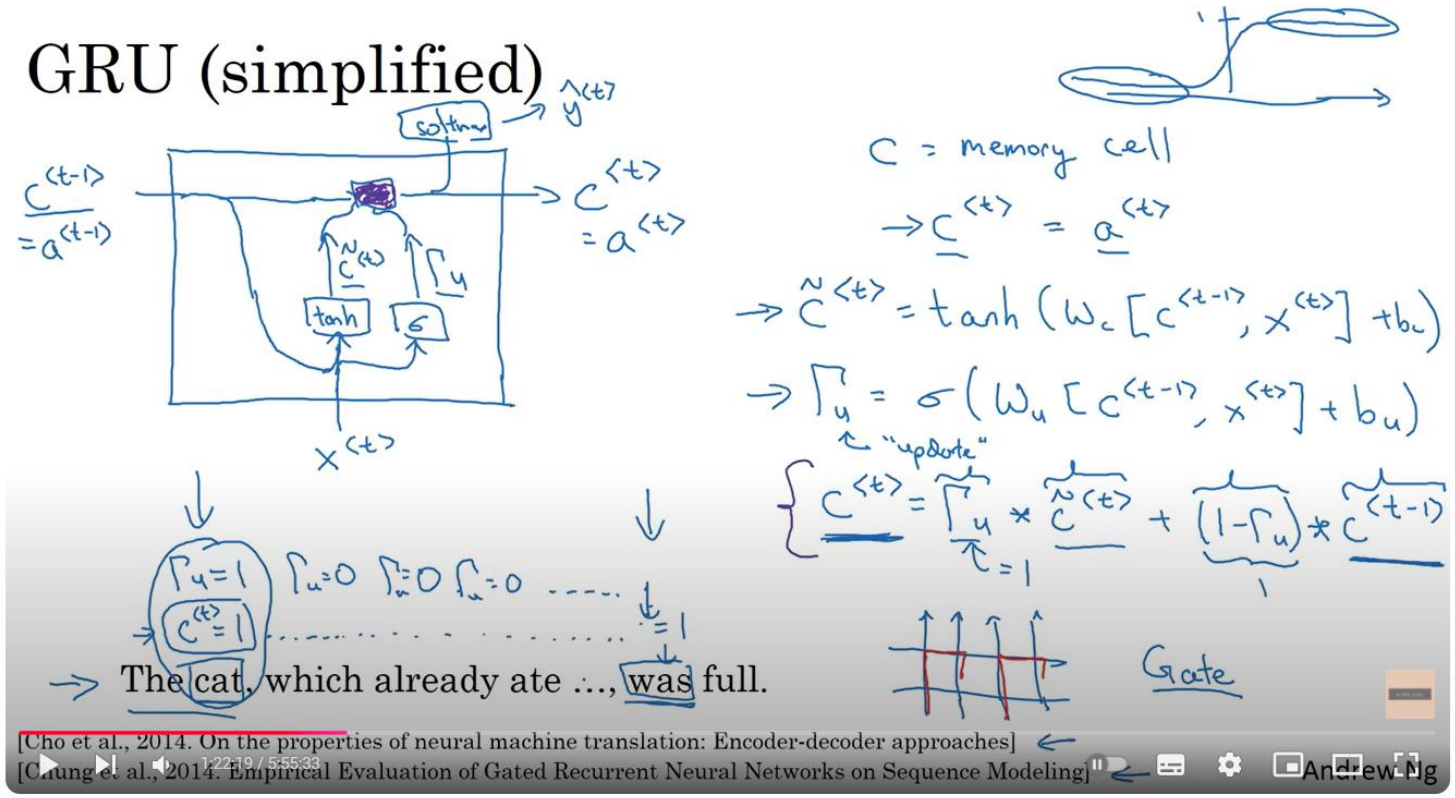
list = ["cat", "dog", "bird", "dog", "cat"]

1. **Identify Unique Categories**:
   - Unique categories are `["cat", "dog", "bird"]`.
2. **Assign Each Category an Index**:
   - "cat" -> 0
   - "dog" -> 1
   - "bird" -> 2
3. **Convert Each Element to a One-Hot Vector**:
   - Use the category's index to create a binary vector with a `1` at that index and `0` elsewhere:
     - "cat" -> `[1, 0, 0]`
     - "dog" -> `[0, 1, 0]`
     - "bird" -> `[0, 0, 1]`

Application: text generation

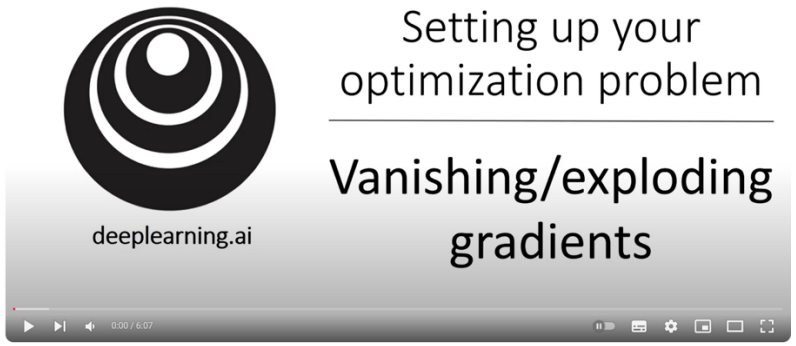


3) Gated recurrent units (GRU)



Sequence Models Complete Course

- RNNs may suffer from the gradients issues (**vanishing/exploding gradients**):



Vanishing/Exploding Gradients (C2W1L10)

- to solve this problems, we have **GRU** (Gated Recurrent Units) and **LSTM** (Long Short-Term Memory)
- **GRU = RNN + *reset gate* + *update gate***

- the ***reset gate*** determines how much of the previous hidden state (h_{t-1}) should be forgotten or ignored when computing the new hidden state $(\tilde{\mathbf{h}}_t)$

- the ***update gate*** controls how much of the new hidden state $(\tilde{\mathbf{h}}_t)$ and the previous hidden state (h_{t-1}) should be combined to form the current hidden state (h_t)

The **tanh** (hyperbolic tangent) function is an activation function applied element-wise to a matrix. The formula for $\tanh(x)$ is:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$\begin{cases} \mathbf{R_t} = \sigma(\mathbf{X_t W_{xr}} + \mathbf{H_{t-1} W_{hr}} + \mathbf{b_r}) \\ \mathbf{Z_t} = \sigma(\mathbf{X_t W_{xz}} + \mathbf{H_{t-1} W_{hz}} + \mathbf{b_z}), \\ \tilde{\mathbf{H}}_t = tanh(\mathbf{X_t W_{xh}} + (\mathbf{R_t} \odot \mathbf{H_{t-1}})\mathbf{W_{hh}} + \mathbf{b_h}) \\ \mathbf{H_t} = \mathbf{Z_t} \odot \mathbf{H_{t-1}} + (1 - \mathbf{Z_t}) \odot \tilde{\mathbf{H}}_t \end{cases}$$

INLOCUIM TOATE ECUATIILE UNUI GRU CU CEVA MAI SIMPLU

GRU = RNN + *reset gate* + *update gate*

the **Hadamard product** (also known as the element-wise product) between 2 matrices is computed by multiplying corresponding elements of the 2 matrices:

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}.$$

The Hadamard product $\mathbf{A} \odot \mathbf{B}$ is computed as:

$$\mathbf{A} \odot \mathbf{B} = \begin{bmatrix} 1 \cdot 5 & 2 \cdot 6 \\ 3 \cdot 7 & 4 \cdot 8 \end{bmatrix} = \begin{bmatrix} 5 & 12 \\ 21 & 32 \end{bmatrix}$$

NOTE: comparing with a RNN layer, now for a GRU layer we will have the following learnable parameters: W_{xr} , W_{hr} , W_{xz} , W_{hz} , W_{xh} , W_{hh} , b_r , b_z , b_h

RNN

$$\mathbf{H_t} = \phi(\mathbf{X_t}\mathbf{W_{xh}} + \mathbf{H_{t-1}}\mathbf{W_{hh}} + \mathbf{b_h})$$

```
rnn_layer = nn.RNN(num_inputs, num_hiddens)
model = RNNModel(rnn_layer, len(vocab))
```

GRU

$$\mathbf{R_t} = \sigma(\mathbf{X_t}\mathbf{W_{xr}} + \mathbf{H_{t-1}}\mathbf{W_{hr}} + \mathbf{b_r})$$

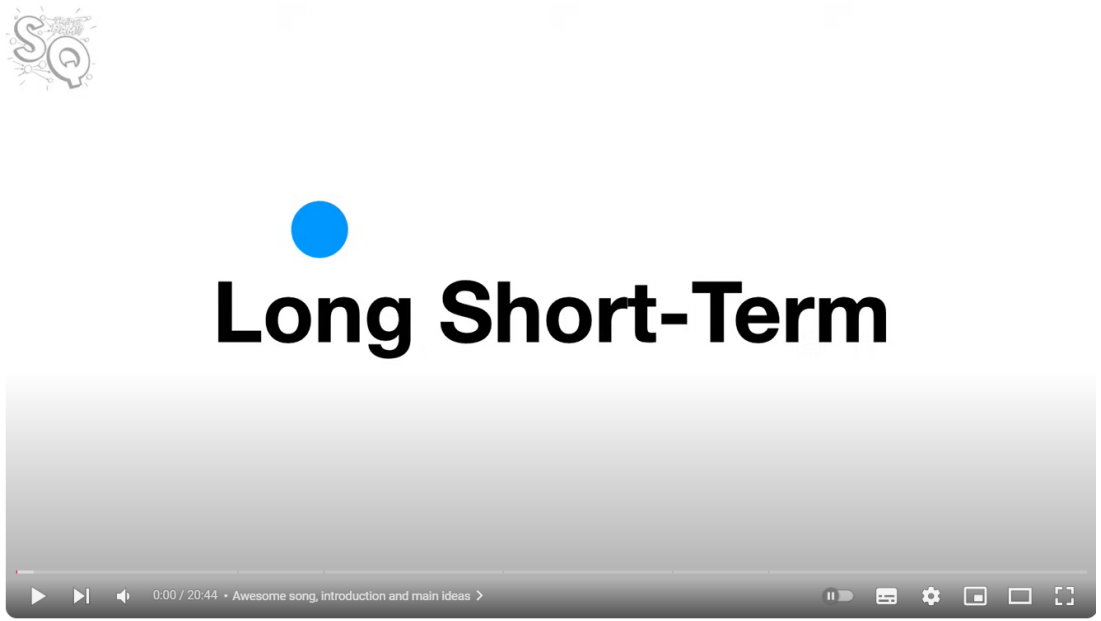
$$\mathbf{Z_t} = \sigma(\mathbf{X_t}\mathbf{W_{xz}} + \mathbf{H_{t-1}}\mathbf{W_{hz}} + \mathbf{b_z}),$$

$$\tilde{\mathbf{H}}_\mathbf{t} = tanh(\mathbf{X_t}\mathbf{W_{xh}} + (\mathbf{R_t} \odot \mathbf{H_{t-1}})\mathbf{W_{hh}} + \mathbf{b_h})$$

$$\mathbf{H_t} = \mathbf{Z_t} \odot \mathbf{H_{t-1}} + (1 - \mathbf{Z_t}) \odot \tilde{\mathbf{H}}_\mathbf{t}$$

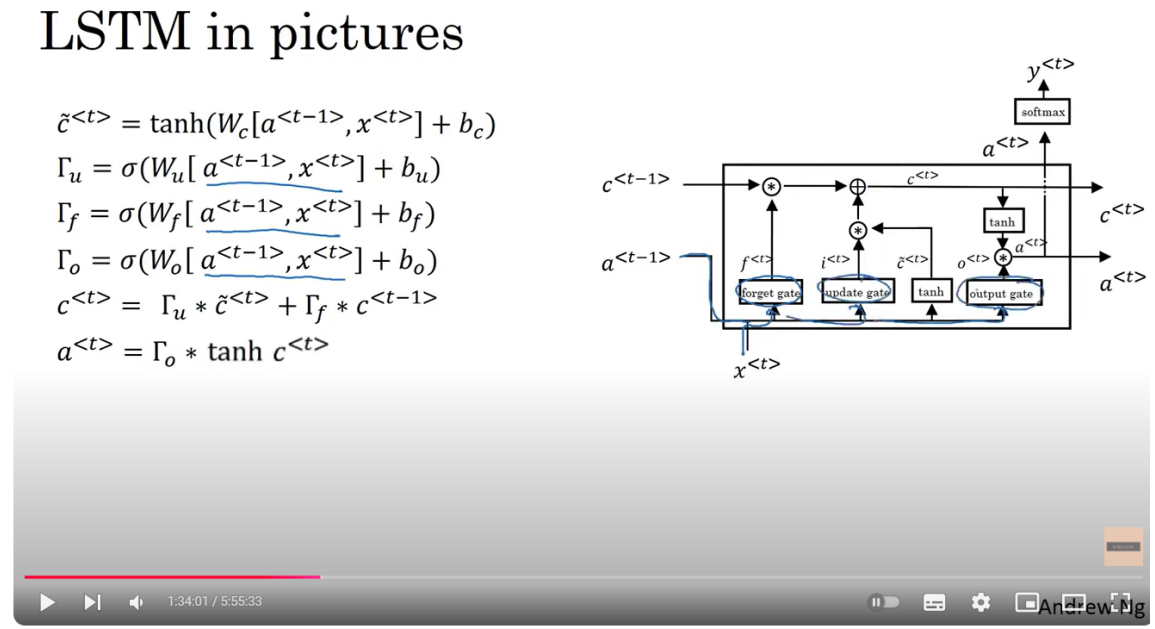
```
gru_layer = nn.GRU(num_inputs, num_hiddens)
model = RNNModel(gru_layer, len(vocab))
```

4) Long short-term memory (LSTM)



Long Short-Term Memory (LSTM), Clearly Explained

StatQuest with Josh Starmer
1,29 mil. de abonați
Alătură-te
Abonează-te
15 K
Trimite
Descarcă
Mulțumire



Sequence Models Complete Course

$$
\begin{cases}
\mathbf{I_t} = \sigma(\mathbf{X_t W_{xi}} + \mathbf{H_{t-1} W_{hi}} + \mathbf{b_i}) \\
\mathbf{F_t} = \sigma(\mathbf{X_t W_{xf}} + \mathbf{H_{t-1} W_{hf}} + \mathbf{b_f}) \\
\mathbf{O_t} = \sigma(\mathbf{X_t W_{xo}} + \mathbf{H_{t-1} W_{ho}} + \mathbf{b_o}) \\
\tilde{\mathbf{C}}_\mathbf{t} = tanh(\mathbf{X_t W_{xc}} + \mathbf{H_{t-1} W_{hc}} + \mathbf{b_c}) \\
\mathbf{C_t} = \mathbf{F_t} \odot \mathbf{C_{t-1}} + \mathbf{I_t} \odot \tilde{\mathbf{C}}_\mathbf{t} \\
\mathbf{H_t} = \mathbf{O_t} \odot tanh(\mathbf{C_t})
\end{cases}
$$

the **Hadamard product** (also known as the element-wise product) between 2 matrices is computed by multiplying corresponding elements of the 2 matrices:

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}.$$

The Hadamard product $\mathbf{A} \odot \mathbf{B}$ is computed as:

$$\mathbf{A} \odot \mathbf{B} = \begin{bmatrix} 1 \cdot 5 & 2 \cdot 6 \\ 3 \cdot 7 & 4 \cdot 8 \end{bmatrix} = \begin{bmatrix} 5 & 12 \\ 21 & 32 \end{bmatrix}$$

The **tanh** (hyperbolic tangent) function is an activation function applied element-wise to a matrix. The formula for $\tanh(x)$ is:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

INLOCUIM TOATE ECUATIILE UNUI LSTM CU CEVA MAI SIMPLU

LSTM = RNN + *output gate* + *input gate* + *forget gate* + *cell state*

RNN

$$\mathbf{H}_t = \phi(\mathbf{X}_t \mathbf{W}_{xh} + \mathbf{H}_{t-1} \mathbf{W}_{hh} + \mathbf{b}_h)$$

```
rnn_layer = nn.RNN(num_inputs, num_hiddens)
model = RNNModel(rnn_layer, len(vocab))
```

GRU

$$\mathbf{R}_t = \sigma(\mathbf{X}_t \mathbf{W}_{xr} + \mathbf{H}_{t-1} \mathbf{W}_{hr} + \mathbf{b}_r)$$

$$\mathbf{Z}_t = \sigma(\mathbf{X}_t \mathbf{W}_{xz} + \mathbf{H}_{t-1} \mathbf{W}_{hz} + \mathbf{b}_z),$$

$$\tilde{\mathbf{H}}_t = tanh(\mathbf{X}_t \mathbf{W}_{xh} + (\mathbf{R}_t \odot \mathbf{H}_{t-1}) \mathbf{W}_{hh} + \mathbf{b}_h)$$

$$\mathbf{H}_t = \mathbf{Z}_t \odot \mathbf{H}_{t-1} + (1 - \mathbf{Z}_t) \odot \tilde{\mathbf{H}}_t$$

```
gru_layer = nn.GRU(num_inputs, num_hiddens)
model = RNNModel(gru_layer, len(vocab))
```

LSTM

$$\mathbf{I}_t = \sigma(\mathbf{X}_t \mathbf{W}_{xi} + \mathbf{H}_{t-1} \mathbf{W}_{hi} + \mathbf{b}_i)$$

$$\mathbf{F}_t = \sigma(\mathbf{X}_t \mathbf{W}_{xf} + \mathbf{H}_{t-1} \mathbf{W}_{hf} + \mathbf{b}_f)$$

$$\mathbf{O}_t = \sigma(\mathbf{X}_t \mathbf{W}_{xo} + \mathbf{H}_{t-1} \mathbf{W}_{ho} + \mathbf{b}_o)$$

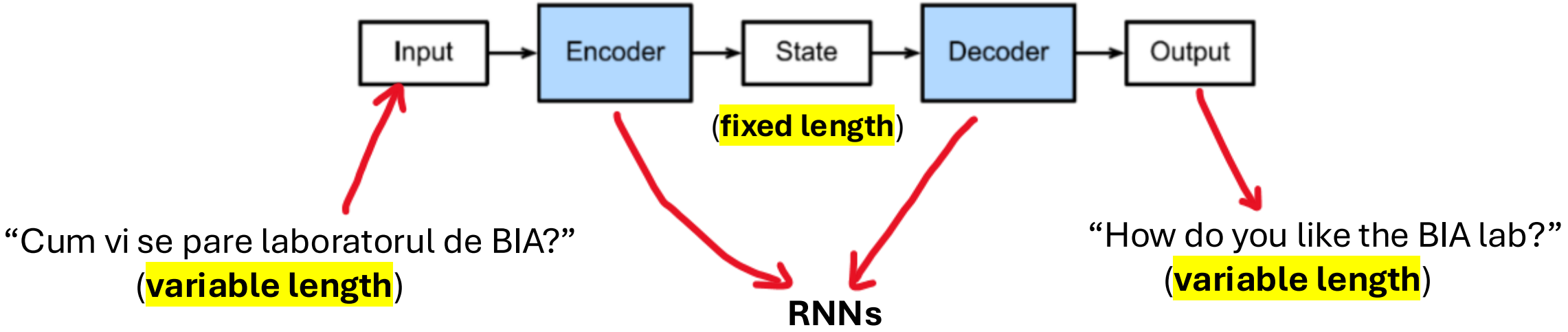
$$\tilde{\mathbf{C}}_t = tanh(\mathbf{X}_t \mathbf{W}_{xc} + \mathbf{H}_{t-1} \mathbf{W}_{hc} + \mathbf{b}_c)$$

$$\mathbf{C}_t = \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{C}}_t$$

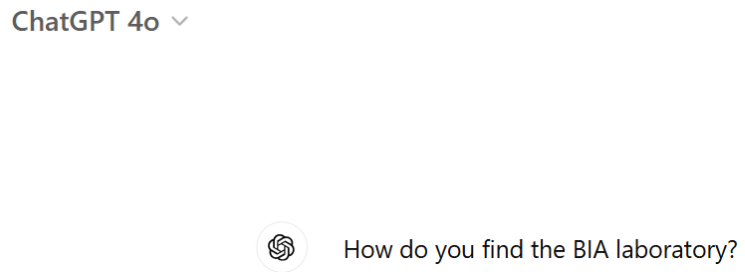
$$\mathbf{H}_t = \mathbf{O}_t \odot tanh(\mathbf{C}_t)$$

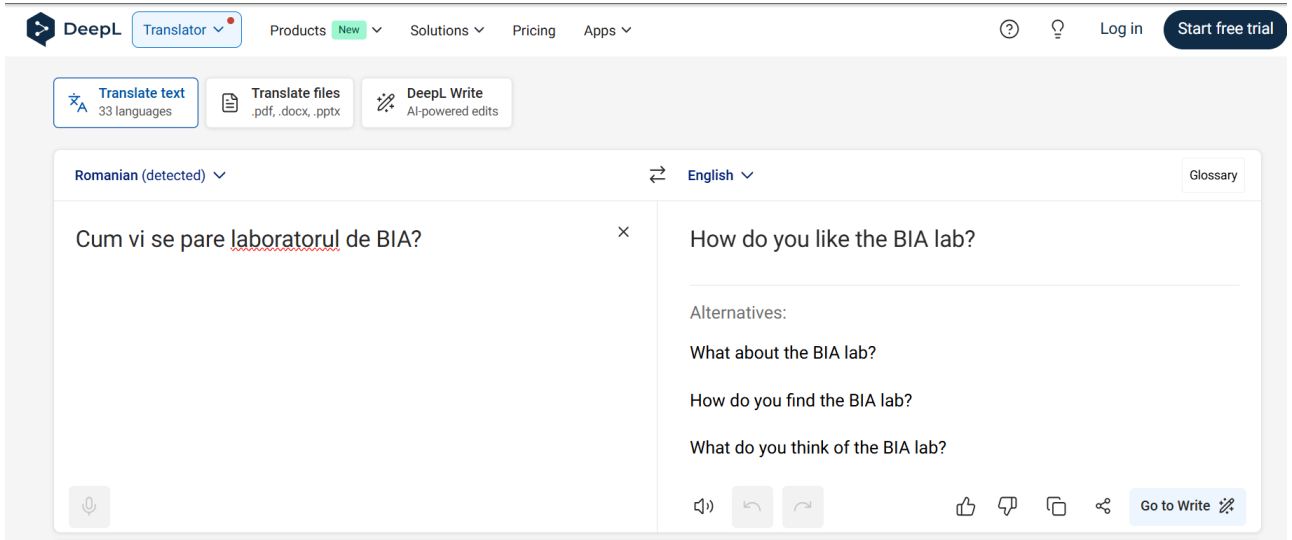
```
lstm_layer = nn.LSTM(num_inputs, num_hiddens)
model = RNNModel(lstm_layer, len(vocab))
```

5) Sequence to sequence learning



Applications:





Dataset

- English-French dataset (we want to translate from English to French)

```python
def download_extract(url, folder=None):
    """Download and extract a zip file."""
    fname = download(url)
    base_dir = os.path.dirname(fname)
    data_dir, ext = os.path.splitext(fname)
    if ext == '.zip':
        fp = zipfile.ZipFile(fname, 'r')
    else:
        assert False, 'Only zip files can be extracted.'
    fp.extractall(base_dir)
    return os.path.join(base_dir, folder) if folder else data_dir
```

```python
def read_data_nmt():
    """Load the English-French dataset."""
    data_dir = download_extract('http://d2l-data.s3-accelerate.amazonaws.com/fra-eng.zip')
    with open(os.path.join(data_dir, 'fra.txt'), 'r') as f:
        return f.read()

raw_text = read_data_nmt()
print(raw_text[:75])
```

```
Go.     Va !
Hi.     Salut !
Run!    Cours !
Run!    Courez !
Who?    Qui ?
Wow!    Ça alors !
```

+ other more functions to process the dataset....

Model

- 2 RNNs: one for encoder and one for decoder:

```python
class Encoder(nn.Module):
    """The base encoder interface for the encoder-decoder architecture."""
    def __init__(self, **kwargs):
        super(Encoder, self).__init__(**kwargs)

    def forward(self, X, *args):
        raise NotImplementedError
```

```python
class Seq2SeqEncoder(Encoder):
    """The RNN encoder for sequence to sequence learning."""
    def __init__(self, vocab_size, embed_size, num_hiddens, num_layers,
                 dropout=0, **kwargs):
        super(Seq2SeqEncoder, self).__init__(**kwargs)
        # Embedding layer
        self.embedding = nn.Embedding(vocab_size, embed_size)
        self.rnn = nn.GRU(embed_size, num_hiddens, num_layers,
                          dropout=dropout)

    def forward(self, X, *args):
        # The output `X` shape: (`batch_size`, `num_steps`, `embed_size`)
        X = self.embedding(X)
        # In RNN models, the first axis corresponds to time steps
        X = X.permute(1, 0, 2)
        # When state is not mentioned, it defaults to zeros
        output, state = self.rnn(X)
        # `output` shape: (`num_steps`, `batch_size`, `num_hiddens`)
        # `state` shape: (`num_layers`, `batch_size`, `num_hiddens`)
        return output, state
```

```python
class Decoder(nn.Module):
    """The base decoder interface for the encoder-decoder architecture."""
    def __init__(self, **kwargs):
        super(Decoder, self).__init__(**kwargs)

    def init_state(self, enc_outputs, *args):
        raise NotImplementedError

    def forward(self, X, state):
        raise NotImplementedError
```

```python
class Seq2SeqDecoder(Decoder):
    """The RNN decoder for sequence to sequence learning."""
    def __init__(self, vocab_size, embed_size, num_hiddens, num_layers,
                 dropout=0, **kwargs):
        super(Seq2SeqDecoder, self).__init__(**kwargs)
        self.embedding = nn.Embedding(vocab_size, embed_size)
        self.rnn = nn.GRU(embed_size + num_hiddens, num_hiddens, num_layers,
                          dropout=dropout)
        self.dense = nn.Linear(num_hiddens, vocab_size)

    def init_state(self, enc_outputs, *args):
        return enc_outputs[1]

    def forward(self, X, state):
        # The output `X` shape: (`num_steps`, `batch_size`, `embed_size`)
        X = self.embedding(X).permute(1, 0, 2)
        # Broadcast `context` so it has the same `num_steps` as `X`
        context = state[-1].repeat(X.shape[0], 1, 1)
        X_and_context = torch.cat((X, context), 2)
        output, state = self.rnn(X_and_context, state)
        output = self.dense(output).permute(1, 0, 2)
        # `output` shape: (`batch_size`, `num_steps`, `vocab_size`)
        # `state` shape: (`num_layers`, `batch_size`, `num_hiddens`)
        return output, state
```

```python
class EncoderDecoder(nn.Module):
    """The base class for the encoder-decoder architecture."""
    def __init__(self, encoder, decoder, **kwargs):
        super(EncoderDecoder, self).__init__(**kwargs)
        self.encoder = encoder
        self.decoder = decoder

    def forward(self, enc_X, dec_X, *args):
        enc_outputs = self.encoder(enc_X, *args)
        dec_state = self.decoder.init_state(enc_outputs, *args)
        return self.decoder(dec_X, dec_state)
```

these are specified in the problem

```python
encoder = Seq2SeqEncoder(len(src_vocab), embed_size, num_hiddens, num_layers, dropout)
decoder = Seq2SeqDecoder(len(tgt_vocab), embed_size, num_hiddens, num_layers, dropout)
net = EncoderDecoder(encoder, decoder)
```

this is the model used for training ☺
we need to use 5 classes to obtain it

Exercises

Laboratory 10 – Exercises

1. Create an RNN with a single recurrent layer, an input size of 2 and a hidden size of 4. The input and the initial hidden state are tensors filled with random numbers of shape `(seq_length, batch, input_size)=(5,1,2)` and `(num_layers · num_directions, batch, hidden_size)=(1,1,4)`, respectively. Print the output of the RNN.

2. Use the text of H. G. Wells' "The Time Machine" to generate 20 characters, starting from the text "traveller". Use an RNN with two recurrent layers, each having 32 hidden units. Train the model for 200 epochs, with a learning rate of 1.5, a batch size of 30, and a number of steps of 10. Print the perplexity of the model at the last epoch.

3. Use the text of H. G. Wells' "The Time Machine" to generate 35 characters, starting from the text "time". Use a GRU with three recurrent layers, each having 16 hidden units. Train the model for 300 epochs, with a learning rate of 3, a batch size of 50, and a number of steps of 15. Print the perplexity of the model at the last epoch.

Solutions

1. Create an RNN with a single recurrent layer, an input size of 2 and a hidden size of 4. The input and the initial hidden state are tensors filled with random numbers of shape (seq_length, batch, input_size)=(5,1,2) and (num_layers · num_directions, batch, hidden_size)=(1,1,4), respectively. Print the output of the RNN.

```python
import hashlib
import os
import requests
import re
import collections
import random
import torch
from torch import nn
from torch.nn import functional as F
import math
import matplotlib.pyplot as plt
import zipfile

torch.manual_seed(42);
```

```python
# Create the RNN
rnn = nn.RNN(input_size=2, hidden_size = 4, num_layers=1)

# Create input tensor with shape (seq_length, batch_size, input_size)
input = torch.rand(5, 1, 2)

# Create initial hidden state tensor with shape (num_layers, batch_size, hidden_size)
h0 = torch.rand(1, 1, 4)

# Forward pass through the RNN
output, hn = rnn(input, h0)

# Print the output and the final hidden state
print("Output of the RNN:")
print(output)
print("\nFinal Hidden State:")
print(hn)
```

```
Output of the RNN:
tensor([[[-0.6658, -0.4526,  0.6346, -0.8986]],

        [[ 0.7734,  0.6522,  0.6107, -0.5843]],

        [[-0.2533, -0.0527,  0.3770, -0.8330]],

        [[ 0.3693,  0.1756,  0.4692, -0.4902]],

        [[-0.2211, -0.0279,  0.4767, -0.6544]]], grad_fn=<StackBackward0>)

Final Hidden State:
tensor([[[-0.2211, -0.0279,  0.4767, -0.6544]]], grad_fn=<StackBackward0>)
```

2. Use the text of H. G. Wells' "The Time Machine" to generate 20 characters, starting from the text "traveller". Use an RNN with two recurrent layers, each having 32 hidden units. Train the model for 200 epochs, with a learning rate of 1.5, a batch size of 30, and a number of steps of 10. Print the perplexity of the model at the last epoch.

- load and prepare the data:

```python
def download(url, cache_dir=os.path.join('..', 'data')):
    """Download a file, return the local filename."""
    os.makedirs(cache_dir, exist_ok=True)
    fname = os.path.join(cache_dir, url.split('/')[-1])
    if os.path.exists(fname):
        with open(fname, 'rb') as f:
            while True:
                data = f.read(1048576)
                if not data:
                    break
        return fname
    print(f'Downloading {fname} from {url}...')
    r = requests.get(url, stream=True, verify=True)
    with open(fname, 'wb') as f:
        f.write(r.content)
    return fname
```

```python
# Step 1
def read_time_machine():
    """Load the time machine dataset into a list of text lines."""
    with open(download('http://d2l-data.s3-accelerate.amazonaws.com/timemachine.txt'), 'r') as f:
        lines = f.readlines()
    return [re.sub('[^A-Za-z]+', ' ', line).strip().lower() for line in lines]
```

```python
# Step 2
def tokenize(lines, token='word'):
    """Split text lines into word or character tokens."""
    if token == 'word':
        return [line.split() for line in lines]
    elif token == 'char':
        return [list(line) for line in lines]
    else:
        print('ERROR: unknown token type: ' + token)
```

JUST COPY PASTE FROM THE LAB FOR THESE FUNCTIONS!

```python
# Step 3
class Vocab:
    """Vocabulary for text."""
    def __init__(self, tokens=None, min_freq=0, reserved_tokens=None):
        if tokens is None:
            tokens = []
        if reserved_tokens is None:
            reserved_tokens = []
        # Sort according to frequencies
        counter = count_corpus(tokens)
        self._token_freqs = sorted(counter.items(), key=lambda x: x[1],
                                   reverse=True)
        # The index for the unknown token is 0
        self.idx_to_token = ['<unk>'] + reserved_tokens
        self.token_to_idx = {token: idx
                             for idx, token in enumerate(self.idx_to_token)}
        for token, freq in self._token_freqs:
            if freq < min_freq:
                break
            if token not in self.token_to_idx:
                self.idx_to_token.append(token)
                self.token_to_idx[token] = len(self.idx_to_token) - 1

    def __len__(self):
        return len(self.idx_to_token)

    def __getitem__(self, tokens):
        if not isinstance(tokens, (list, tuple)):
            return self.token_to_idx.get(tokens, self.unk)
        return [self.__getitem__(token) for token in tokens]

    def to_tokens(self, indices):
        if not isinstance(indices, (list, tuple)):
            return self.idx_to_token[indices]
        return [self.idx_to_token[index] for index in indices]

    @property
    def unk(self):  # Index for the unknown token
        return 0

    @property
    def token_freqs(self):  # Token frequencies
        return self._token_freqs

def count_corpus(tokens):
    """Count token frequencies."""
    # Here `tokens` is a 1D list or 2D list
    if len(tokens) == 0 or isinstance(tokens[0], list):
        # Flatten a list of token lists into a list of tokens
        tokens = [token for line in tokens for token in line]
    return collections.Counter(tokens)
```

JUST COPY PASTE FROM THE LAB FOR THESE FUNCTIONS!

```python
# Step 4
def load_corpus_time_machine(max_tokens=-1):
    """Return token indices and the vocabulary of the time machine dataset."""
    lines = read_time_machine()
    tokens = tokenize(lines, 'char')
    vocab = Vocab(tokens)
    # Since each text line in the time machine dataset is not necessarily a
    # sentence or a paragraph, flatten all the text lines into a single list
    corpus = [vocab[token] for line in tokens for token in line]
    if max_tokens > 0:
        corpus = corpus[:max_tokens]
    return corpus, vocab

corpus, vocab = load_corpus_time_machine()
```

```python
def seq_data_iter_sequential(corpus, batch_size, num_steps):
    """Generate a mini-batch of subsequences using sequential partitioning."""
    # Start with a random offset to partition a sequence
    offset = random.randint(0, num_steps)
    num_tokens = ((len(corpus) - offset - 1) // batch_size) * batch_size
    Xs = torch.tensor(corpus[offset: offset + num_tokens])
    Ys = torch.tensor(corpus[offset + 1: offset + 1 + num_tokens])
    Xs, Ys = Xs.reshape(batch_size, -1), Ys.reshape(batch_size, -1)
    num_batches = Xs.shape[1] // num_steps
    for i in range(0, num_steps * num_batches, num_steps):
        X = Xs[:, i: i + num_steps]
        Y = Ys[:, i: i + num_steps]
        yield X, Y

class SeqDataLoader:
    """An iterator to load sequence data."""
    def __init__(self, batch_size, num_steps, max_tokens):
        self.corpus, self.vocab = load_corpus_time_machine(max_tokens)
        self.batch_size, self.num_steps = batch_size, num_steps

    def __iter__(self):
        return seq_data_iter_sequential(self.corpus, self.batch_size, self.num_steps)

def load_data_time_machine(batch_size, num_steps, max_tokens=10000):
    """Return the iterator and the vocabulary of the time machine dataset."""
    data_iter = SeqDataLoader(
        batch_size, num_steps, max_tokens)
    return data_iter, data_iter.vocab
```

JUST COPY PASTE FROM THE LAB FOR THESE FUNCTIONS!

```python
batch_size, num_steps = 30, 10
train_iter, vocab = load_data_time_machine(batch_size, num_steps)
```

This is our training data! We had 3 slides of copy-paste to obtain these!

- define the RNN we will use:

```python
num_hiddens = 32
rnn_layer = nn.RNN(len(vocab), num_hiddens, num_layers=2)
```

```python
class RNNModel(nn.Module):
    """The RNN model."""
    def __init__(self, rnn_layer, vocab_size, **kwargs):
        super(RNNModel, self).__init__(**kwargs)
        self.rnn = rnn_layer
        self.vocab_size = vocab_size
        self.num_hiddens = self.rnn.hidden_size
        self.linear = nn.Linear(self.num_hiddens, self.vocab_size)

    def forward(self, inputs, state):
        X = F.one_hot(inputs.T.long(), self.vocab_size)
        X = X.to(torch.float32)
        Y, state = self.rnn(X, state)
        # The fully connected layer will first change the shape of `Y` to
        # (`num_steps` * `batch_size`, `num_hiddens`). Its output shape is
        # (`num_steps` * `batch_size`, `vocab_size`).
        output = self.linear(Y.reshape((-1, Y.shape[-1])))
        return output, state

    def begin_state(self, device, batch_size=1):
        if not isinstance(self.rnn, nn.LSTM):
            # `nn.GRU` takes a tensor as hidden state
            return  torch.zeros((self.rnn.num_layers,
                                 batch_size, self.num_hiddens),
                                 device=device)
        else:
            # `nn.LSTM` takes a tuple of hidden states
            return (torch.zeros((
                self.rnn.num_layers,
                batch_size, self.num_hiddens), device=device),
                    torch.zeros((
                        self.rnn.num_layers,
                        batch_size, self.num_hiddens), device=device))
```

JUST COPY PASTE FROM THE LAB FOR THESE FUNCTIONS!

```python
def try_gpu(i=0):
    """Return gpu(i) if exists, otherwise return cpu()."""
    if torch.cuda.device_count() >= i + 1:
        return torch.device(f'cuda:{i}')
    return torch.device('cpu')

device = try_gpu()
net = RNNModel(rnn_layer, vocab_size=len(vocab))
net = net.to(device)
```

- write the functions for training:

```python
def predict(prefix, num_preds, net, vocab, device):
    """Generate new characters following the `prefix`."""
    state = net.begin_state(batch_size=1, device=device)
    outputs = [vocab[prefix[0]]]
    get_input = lambda: torch.tensor([outputs[-1]], device=device).reshape((1, 1))
    for y in prefix[1:]:  # Warm-up period
        _, state = net(get_input(), state)
        outputs.append(vocab[y])
    for _ in range(num_preds):  # Predict `num_preds` steps
        y, state = net(get_input(), state)
        outputs.append(int(y.argmax(dim=1).reshape(1)))
    return ''.join([vocab.idx_to_token[i] for i in outputs])

def grad_clipping(net, theta):
    """Clip the gradient."""
    params = [p for p in net.parameters() if p.requires_grad]
    norm = torch.sqrt(sum(torch.sum((p.grad ** 2)) for p in params))
    if norm > theta:
        for param in params:
            param.grad[:] *= theta / norm
```

JUST COPY PASTE FROM THE LAB FOR THESE FUNCTIONS!

```python
def train_epoch(net, train_iter, loss, optimizer, device):
    """Train a net within one epoch."""
    state = None
    # Sum of training loss, no. of tokens
    total_loss = 0
    total_tokens = 0
    for X, Y in train_iter:
        if state is None:
            # Initialize `state` when it is the first iteration
            state = net.begin_state(batch_size=X.shape[0], device=device)
        else:
            if not isinstance(state, tuple):
                # `state` is a tensor for `nn.GRU`
                state.detach_()
            else:
                # `state` is a tuple of tensors for `nn.LSTM`
                for s in state:
                    s.detach_()
        y = Y.T.reshape(-1)
        X, y = X.to(device), y.to(device)
        y_hat, state = net(X, state)
        l = loss(y_hat, y.long()).mean()
        optimizer.zero_grad()
        l.backward()
        grad_clipping(net, 1)
        optimizer.step()
        total_loss += float(l * y.numel())
        total_tokens += y.numel()
    return math.exp(total_loss / total_tokens)

def train(net, train_iter, vocab, lr, num_epochs, device):
    """Train a model."""
    loss = nn.CrossEntropyLoss()
    perplexities = []
    # Initialize
    optimizer = torch.optim.SGD(net.parameters(), lr)
    # Train and predict
    for epoch in range(num_epochs):
        ppl = train_epoch(
            net, train_iter, loss, optimizer, device)
        if (epoch + 1) % 10 == 0:
            print(predict('traveller', 20, net, vocab, device))
            perplexities.append(ppl)
    print(f'perplexity {ppl:.1f}, device {str(device)}')
    print(predict('traveller', 20, net, vocab, device))
    print(predict('traveller', 20, net, vocab, device))

    return perplexities
```

I changed only these ☺

- train the model:

```
num_epochs, lr = 200, 1.5
perplexities = train(net, train_iter, vocab, lr, num_epochs, device)
```

```
travelleris theredions and an
traveller so the think and an
traveller a tome thing the ti
traveller and the pouthe got
traveller and and the sat the
traveller the gat the gat the
traveller and the time said t
traveller the thing the this
traveller a momethere the som
traveller a for a for a for a
traveller a line a line some
traveller and and and and and
traveller and the pre and the
traveller a line a look of th
traveller a look the time tra
traveller the time travel ot
traveller and the time track
traveller a loone we with a m
traveller a move for the time
traveller the this the this t
perplexity 5.0, device cpu
traveller the this the this t
traveller the this the this t
```

- print the perplexity of the model at the last epoch:

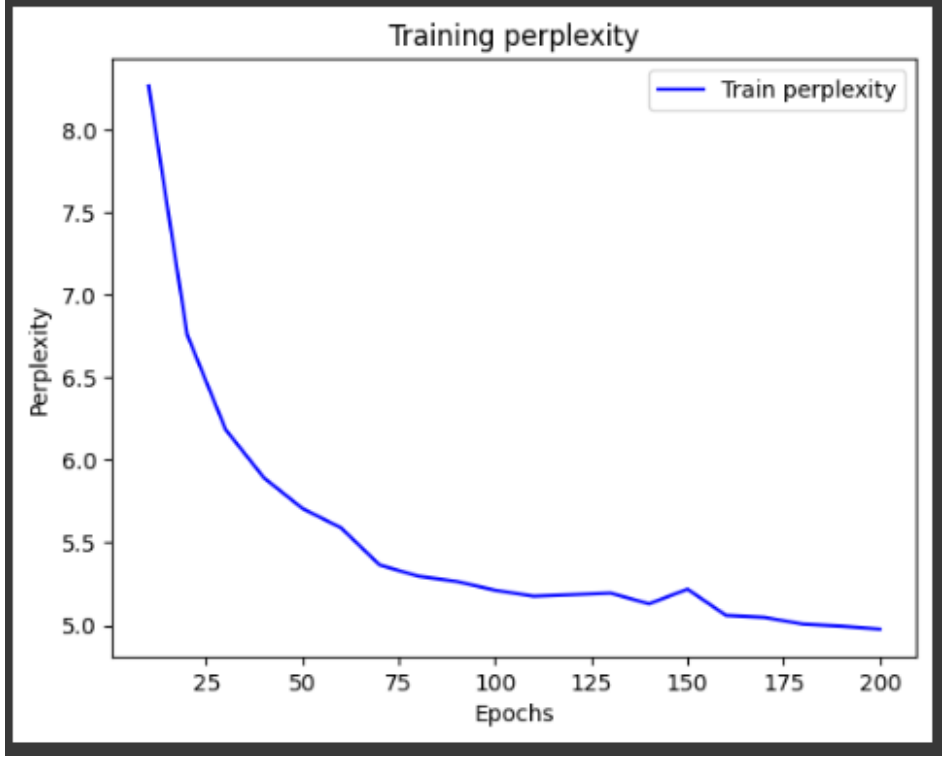
```
print(perplexities[-1])
```

```
4.977019031392368
```

- plot the perplexities obtained during training:

```
def plot_perplexity(perplexities):
    epochs = range(10, len(perplexities * 10) + 1, 10)
    plt.plot(epochs, perplexities, 'b', label='Train perplexity')
    plt.title('Training perplexity')
    plt.xlabel('Epochs')
    plt.ylabel('Perplexity')
    plt.legend()
    plt.show()
```

```
plot_perplexity(perplexities)
```



3. Use the text of H. G. Wells' "The Time Machine" to generate 35 characters, starting from the text "time". Use a GRU with three recurrent layers, each having 16 hidden units. Train the model for 300 epochs, with a learning rate of 3, a batch size of 50, and a number of steps of 15. Print the perplexity of the model at the last epoch.

- load and prepare the data:

```python
def download(url, cache_dir=os.path.join('..', 'data')):
    """Download a file, return the local filename."""
    os.makedirs(cache_dir, exist_ok=True)
    fname = os.path.join(cache_dir, url.split('/')[-1])
    if os.path.exists(fname):
        with open(fname, 'rb') as f:
            while True:
                data = f.read(1048576)
                if not data:
                    break
        return fname
    print(f'Downloading {fname} from {url}...')
    r = requests.get(url, stream=True, verify=True)
    with open(fname, 'wb') as f:
        f.write(r.content)
    return fname
```

```python
# Step 1
def read_time_machine():
    """Load the time machine dataset into a list of text lines."""
    with open(download('http://d2l-data.s3-accelerate.amazonaws.com/timemachine.txt'), 'r') as f:
        lines = f.readlines()
    return [re.sub('[^A-Za-z]+', ' ', line).strip().lower() for line in lines]
```

```python
# Step 2
def tokenize(lines, token='word'):
    """Split text lines into word or character tokens."""
    if token == 'word':
        return [line.split() for line in lines]
    elif token == 'char':
        return [list(line) for line in lines]
    else:
        print('ERROR: unknown token type: ' + token)
```

JUST COPY PASTE FROM THE LAB FOR THESE FUNCTIONS!

```python
# Step 3
class Vocab:
    """Vocabulary for text."""
    def __init__(self, tokens=None, min_freq=0, reserved_tokens=None):
        if tokens is None:
            tokens = []
        if reserved_tokens is None:
            reserved_tokens = []
        # Sort according to frequencies
        counter = count_corpus(tokens)
        self._token_freqs = sorted(counter.items(), key=lambda x: x[1],
                                   reverse=True)
        # The index for the unknown token is 0
        self.idx_to_token = ['<unk>'] + reserved_tokens
        self.token_to_idx = {token: idx
                             for idx, token in enumerate(self.idx_to_token)}
        for token, freq in self._token_freqs:
            if freq < min_freq:
                break
            if token not in self.token_to_idx:
                self.idx_to_token.append(token)
                self.token_to_idx[token] = len(self.idx_to_token) - 1

    def __len__(self):
        return len(self.idx_to_token)

    def __getitem__(self, tokens):
        if not isinstance(tokens, (list, tuple)):
            return self.token_to_idx.get(tokens, self.unk)
        return [self.__getitem__(token) for token in tokens]

    def to_tokens(self, indices):
        if not isinstance(indices, (list, tuple)):
            return self.idx_to_token[indices]
        return [self.idx_to_token[index] for index in indices]

    @property
    def unk(self):  # Index for the unknown token
        return 0

    @property
    def token_freqs(self):  # Token frequencies
        return self._token_freqs

def count_corpus(tokens):
    """Count token frequencies."""
    # Here `tokens` is a 1D list or 2D list
    if len(tokens) == 0 or isinstance(tokens[0], list):
        # Flatten a list of token lists into a list of tokens
        tokens = [token for line in tokens for token in line]
    return collections.Counter(tokens)
```

JUST COPY PASTE FROM THE LAB FOR THESE FUNCTIONS!

```python
# Step 4
def load_corpus_time_machine(max_tokens=-1):
    """Return token indices and the vocabulary of the time machine dataset."""
    lines = read_time_machine()
    tokens = tokenize(lines, 'char')
    vocab = Vocab(tokens)
    # Since each text line in the time machine dataset is not necessarily a
    # sentence or a paragraph, flatten all the text lines into a single list
    corpus = [vocab[token] for line in tokens for token in line]
    if max_tokens > 0:
        corpus = corpus[:max_tokens]
    return corpus, vocab

corpus, vocab = load_corpus_time_machine()
```

```python
def seq_data_iter_sequential(corpus, batch_size, num_steps):
    """Generate a mini-batch of subsequences using sequential partitioning."""
    # Start with a random offset to partition a sequence
    offset = random.randint(0, num_steps)
    num_tokens = ((len(corpus) - offset - 1) // batch_size) * batch_size
    Xs = torch.tensor(corpus[offset: offset + num_tokens])
    Ys = torch.tensor(corpus[offset + 1: offset + 1 + num_tokens])
    Xs, Ys = Xs.reshape(batch_size, -1), Ys.reshape(batch_size, -1)
    num_batches = Xs.shape[1] // num_steps
    for i in range(0, num_steps * num_batches, num_steps):
        X = Xs[:, i: i + num_steps]
        Y = Ys[:, i: i + num_steps]
        yield X, Y

class SeqDataLoader:
    """An iterator to load sequence data."""
    def __init__(self, batch_size, num_steps, max_tokens):
        self.corpus, self.vocab = load_corpus_time_machine(max_tokens)
        self.batch_size, self.num_steps = batch_size, num_steps

    def __iter__(self):
        return seq_data_iter_sequential(self.corpus, self.batch_size, self.num_steps)

def load_data_time_machine(batch_size, num_steps, max_tokens=10000):
    """Return the iterator and the vocabulary of the time machine dataset."""
    data_iter = SeqDataLoader(
        batch_size, num_steps, max_tokens)
    return data_iter, data_iter.vocab
```

JUST COPY PASTE FROM THE LAB FOR THESE FUNCTIONS!

```python
batch_size, num_steps = 50, 15
train_iter, vocab = load_data_time_machine(batch_size, num_steps)
```

This is our training data! We had 3 slides of copy-paste to obtain these!

- define the GRU we will use:

```
num_inputs = len(vocab)
num_hiddens = 16
num_layers=3
gru_layer = nn.GRU(num_inputs, num_hiddens, num_layers)
```

```
class RNNModel(nn.Module):
    """The RNN model."""
    def __init__(self, rnn_layer, vocab_size, **kwargs):
        super(RNNModel, self).__init__(**kwargs)
        self.rnn = rnn_layer
        self.vocab_size = vocab_size
        self.num_hiddens = self.rnn.hidden_size
        self.linear = nn.Linear(self.num_hiddens, self.vocab_size)

    def forward(self, inputs, state):
        X = F.one_hot(inputs.T.long(), self.vocab_size)
        X = X.to(torch.float32)
        Y, state = self.rnn(X, state)
        # The fully connected layer will first change the shape of `Y` to
        # (`num_steps` * `batch_size`, `num_hiddens`). Its output shape is
        # (`num_steps` * `batch_size`, `vocab_size`).
        output = self.linear(Y.reshape((-1, Y.shape[-1])))
        return output, state

    def begin_state(self, device, batch_size=1):
        if not isinstance(self.rnn, nn.LSTM):
            # `nn.GRU` takes a tensor as hidden state
            return  torch.zeros((self.rnn.num_layers,
                                 batch_size, self.num_hiddens),
                                 device=device)
        else:
            # `nn.LSTM` takes a tuple of hidden states
            return (torch.zeros((
                self.rnn.num_layers,
                batch_size, self.num_hiddens), device=device),
                    torch.zeros((
                        self.rnn.num_layers,
                        batch_size, self.num_hiddens), device=device))
```

JUST COPY PASTE FROM THE LAB FOR THESE FUNCTIONS!

```
def try_gpu(i=0):
    """Return gpu(i) if exists, otherwise return cpu()."""
    if torch.cuda.device_count() >= i + 1:
        return torch.device(f'cuda:{i}')
    return torch.device('cpu')

device = try_gpu()
net = RNNModel(gru_layer, len(vocab))
net = net.to(device)
```

- write the functions for training:

```python
def predict(prefix, num_preds, net, vocab, device):
    """Generate new characters following the `prefix`."""
    state = net.begin_state(batch_size=1, device=device)
    outputs = [vocab[prefix[0]]]
    get_input = lambda: torch.tensor([outputs[-1]], device=device).reshape((1, 1))
    for y in prefix[1:]:  # Warm-up period
        _, state = net(get_input(), state)
        outputs.append(vocab[y])
    for _ in range(num_preds):  # Predict `num_preds` steps
        y, state = net(get_input(), state)
        outputs.append(int(y.argmax(dim=1).reshape(1)))
    return ''.join([vocab.idx_to_token[i] for i in outputs])

def grad_clipping(net, theta):
    """Clip the gradient."""
    params = [p for p in net.parameters() if p.requires_grad]
    norm = torch.sqrt(sum(torch.sum((p.grad ** 2)) for p in params))
    if norm > theta:
        for param in params:
            param.grad[:] *= theta / norm
```

JUST COPY PASTE FROM THE LAB FOR THESE FUNCTIONS!

```python
def train_epoch(net, train_iter, loss, optimizer, device):
    """Train a net within one epoch."""
    state = None
    # Sum of training loss, no. of tokens
    total_loss = 0
    total_tokens = 0
    for X, Y in train_iter:
        if state is None:
            # Initialize `state` when it is the first iteration
            state = net.begin_state(batch_size=X.shape[0], device=device)
        else:
            if not isinstance(state, tuple):
                # `state` is a tensor for `nn.GRU`
                state.detach_()
            else:
                # `state` is a tuple of tensors for `nn.LSTM`
                for s in state:
                    s.detach_()
        y = Y.T.reshape(-1)
        X, y = X.to(device), y.to(device)
        y_hat, state = net(X, state)
        l = loss(y_hat, y.long()).mean()
        optimizer.zero_grad()
        l.backward()
        grad_clipping(net, 1)
        optimizer.step()
        total_loss += float(l * y.numel())
        total_tokens += y.numel()
    return math.exp(total_loss / total_tokens)

def train(net, train_iter, vocab, lr, num_epochs, device):
    """Train a model."""
    loss = nn.CrossEntropyLoss()
    perplexities = []
    # Initialize
    optimizer = torch.optim.SGD(net.parameters(), lr)
    # Train and predict
    for epoch in range(num_epochs):
        ppl = train_epoch(
            net, train_iter, loss, optimizer, device)
        if (epoch + 1) % 10 == 0:
            print(predict('time', 35, net, vocab, device))
            perplexities.append(ppl)
    print(f'perplexity {ppl:.1f}, device {str(device)}')
    print(predict('time', 35, net, vocab, device))
    print(predict('time', 35, net, vocab, device))

    return perplexities
```

I changed only these ☺

- train the model:

```
num_epochs, lr = 300, 3
perplexities = train(net, train_iter, vocab, lr, num_epochs, device)
```

```
time t a a a a a a a a a a a a a a a
time the the the the the the the the th
time the the the the the the the the th
time the the the the the the the the th
timensions of the the that the that the
time the the the the the the the the th
timension of the time the time the time
time the time the time the time the tim
timension of the the the the the the th
timension on the that the that the that
timensions of spastore the time travell
timensions and and and and and and and
timensions and the time traveller the t
timension one said the psycholofic has
timensions only and and and and and and
timensions on the te traveller of space
timensions of space and the dimensions
time traveller the time traveller the t
time the time the time the time the tim
time there re and and the te the te the
time traveller said the medical the tim
timensions and the light one of the dhi
time traveller have tolle the time trav
timensions of space the time traveller
time the time traveller the time travel
timensions only a move the say and the
timent of the time traveller the time t
timensions it and the light and the lig
timentation said the psycholon of the a
timent of the time traveller the time t
perplexity 4.6, device cpu
timent of the time traveller the time t
timent of the time traveller the time t
```

- print the perplexity of the model at the last epoch:

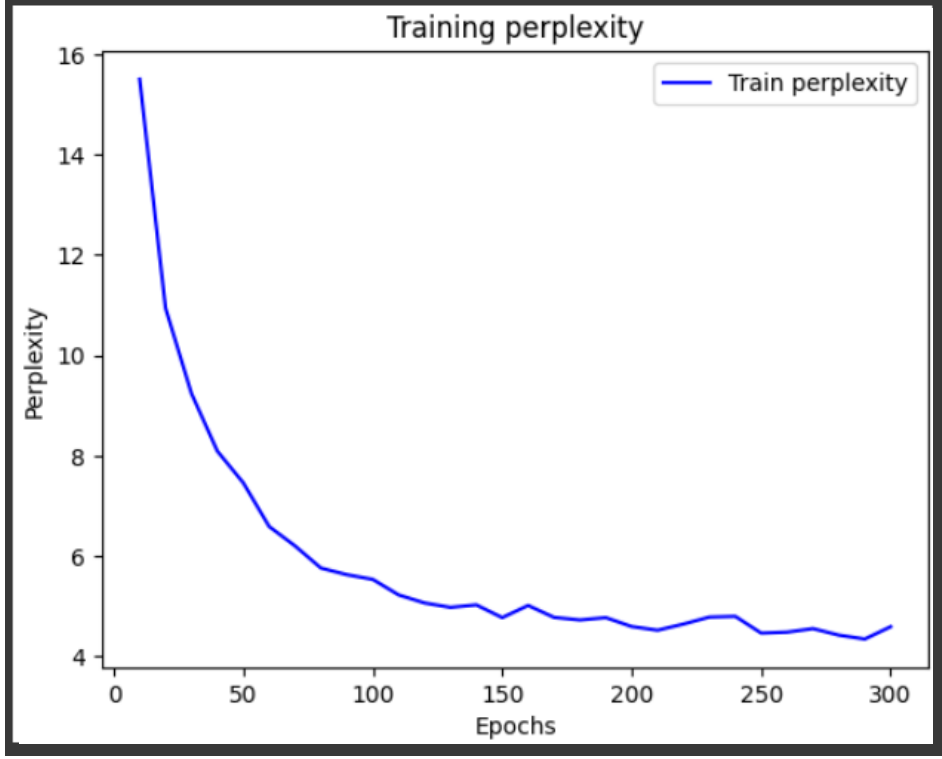
```
print(perplexities[-1])
```

```
4.588687511432917
```

- plot the perplexities obtained during training:

```
def plot_perplexity(perplexities):
    epochs = range(10, len(perplexities * 10) + 1, 10)
    plt.plot(epochs, perplexities, 'b', label='Train perplexity')
    plt.title('Training perplexity')
    plt.xlabel('Epochs')
    plt.ylabel('Perplexity')
    plt.legend()
    plt.show()
```

```
plot_perplexity(perplexities)
```



Thank you !